Software Design Document

Pre-Order Food Delivery Application

Deliverable Lab 4, Steps 1–7 · **Project** Pre-Order Food Delivery Application · **Team** Ali Chamakh · Hamza Bel Labyed · Mouhamed Krid
· **Date** 28 September 2026 · **Standard** IEEE Std 1016–1998

HOW TO READ THIS DOCUMENT

This SDD was written from the Software Requirements Specification and the Requirements Elicitation record, and nothing else. Where those sources make a decision, the decision is designed and traced. Where they do not, the design element is **not invented**: it is marked PROPOSED where the design must take a position to be buildable, and UNRESOLVED where the answer belongs to the stakeholders and no position can be responsibly taken.

That rule is taken from the specification deck itself, which states that an unresolved decision *"must not be invented by an implementation agent."* Every such element is collected in **§9.4** with the open question it depends on. Appendix B shows the design-quality checklist with an honest verdict for each item.

Contents

1 Introduction
1.1 Purpose
1.2 Scope
1.3 Definitions, acronyms and abbreviations
1.4 References
2 System Architecture
2.1 Design methodology and architectural style (Lab 4, Step 1)
2.2 Component diagram
2.3 Component descriptions
2.4 Architectural constraints and layer rules
3 Detailed Design
3.1 Class diagram
3.2 Class descriptions
3.3 Relationships
3.4 Design decisions taken in the class model
3.5 Deliberately absent from the class model
4 Data Design
4.1 Persistent entities
4.2 Integrity rules
4.3 Derived and snapshot values
4.4 Transaction boundaries
5 Interface Design
5.1 User interface overview

5.2 API and integration points
5.3 Authentication and authorisation
5.4 Interfaces not designed
6 Behavioral Design
6.1 Sequence diagrams for key use cases
6.2 Event catalogue
6.3 Behaviour deliberately not designed
7 Design Rationale
7.1 Design patterns used and why
7.2 Alternatives considered and trade-offs
7.3 Design decision register
8 Requirements Traceability Matrix
8.1 Functional requirements to design elements
8.2 Business rules to enforcement points
8.3 Coverage summary
9 Appendices
A Use case register
B Design quality checklist
C Self-review log
D Unresolved design items
E Diagram index

Lab final submission checklist

The seven items below are the final submission checklist of Lab 4, Step 7, in the brief's own words. Each is marked against what this document actually contains, with the section that satisfies it and the page it falls on.

	Final submission item (Step 7)	Status	Where it is answered	Page
☑	Methodology & architecture justification (Step 1)	Included	§2.1 — two comparison tables with the mandated column headings (*Option Considered* | *Fits Your Project?* | *Reason for Choice / Rejection*), a four-sentence justification at §2.1.3, and the microservices revisit triggers at §2.1.4.	6
☑	Architecture diagram + component responsibility table (Step 2)	Included	Figure 1, component and architecture diagram, at §2.2. Component responsibility table — component, layer, responsibility, traced requirements — at §2.3, with architectural constraints at §2.4.	8
☑	Class diagram + class responsibility table (Step 3)	Included	Figure 2, UML class diagram, at §3.1. Class responsibility table at §3.2, relationships at §3.3, and the design decisions taken where the requirements are silent at §3.4.	11
☑	2+ sequence diagrams linked to use cases (Step 4)	Included	Three, not two: Figure 3 (UC-01 place a pre-order), Figure 4 (UC-02 assign and deliver), Figure 5 (UC-03 record payment). All at §6.1. Each figure is linked to a use case identifier from the Appendix A register.	23
☑	Design pattern justification table (Step 5)	Included	The justification table is the opening of §7.1: one row per pattern with its problem, solution, benefit, cost and requirements. The five detailed justifications follow it at §7.1.1 to §7.1.5. Alternatives are in §7.2, decisions in §7.3.	26
☑	Validation checklist + design traceability matrix (Step 6)	Included	Validation checklist at Appendix B — the seven items of Step 6.1, each answered. Traceability matrix at §8, with the mandated columns *Requirement ID* | *Design Element (Component/Class)* | *Notes*: all 30 FRs at §8.1, all 16 business rules at §8.2.	34
☑	Complete SDD document in the standard structure (Step 7)	Included	This document, in the nine-section IEEE 1016 structure prescribed in Step 7, with Steps 1 to 6 folded in and a step-to-section map at Appendix E.	3

TWO DEVIATIONS THE MARKER SHOULD READ BEFORE THE BRIEF

Use case identifiers are derived, not quoted. The brief asks for sequence diagrams "linked to use cases." The Software Requirements Specification lists use cases but does not number them, so a `UC-xx` register was derived from the specification's own interactions and requirement modules, and the derivation is shown in Appendix A. This is recorded as D-11. If the SRS is later given official use case numbers, only Appendix A is renumbered.

Step 6.3 is a self-review, not a peer review. The brief permits "review it yourself after a break." No classmate review has taken place, so Appendix C is labelled a self-review and is not presented as a peer review. It logs six issues found and how each was fixed, against a required minimum of three.

One further item is deliberately *not* marked complete: the seventh row of the Appendix B checklist (non-functional requirements) is recorded as a Partial, and the seven-item final submission checklist above is complete because the item *was delivered and answered* — not because every answer was positive. The reasoning is in Appendix B and §9.4 item U-11.

1Introduction

1.1 Purpose

This document is the software design for the Pre-Order Food Delivery Application. It describes the architecture, the component and class structure, the data model, the interfaces, the behaviour and the design rationale, and it maps every

functional requirement and business rule to the design element that implements it.

It is the Lab 4 deliverable and follows the nine-section structure of IEEE Std 1016–1998 (*Software Design Descriptions*), which is the structure prescribed in the lab brief. It supersedes the Step 1 design-selection note, whose content now appears as §2.1 of this document.

Intended readers: the development team, the reviewer of this lab, and any future maintainer who needs to know which design choices were forced by the requirements and which were open.

1.2 Scope

The system designed here is a coordination platform. It connects a customer who has been registered in person at a restaurant, that restaurant, and a delivery person employed by that same restaurant, so that a food order placed before a morning deadline is prepared and delivered during the lunch period. The problem the system addresses is **congestion at the lunch service period**, not delivery speed.

IN SCOPE

- In-person customer registration and per-restaurant account access `(FR-01 to FR-04, BR-04, BR-05, BR-16)`
- Restaurant and menu management with item-level availability `(FR-05 to FR-08, BR-03)`
- Pre-ordering under a restaurant-defined deadline `(FR-09 to FR-15, BR-01, BR-02)`
- Customer verification against a target organization `(FR-16, FR-17)`
- Restaurant-owned delivery personnel, assignment and delivery confirmation `(FR-18 to FR-24, BR-06, BR-07, BR-10, BR-11)`
- Payment method selection and payment status, cash on delivery and approved third-party Tunisian electronic payment `(FR-25 to FR-28, BR-12, BR-13)`
- Account provisioning for restaurant staff and restaurant-owned delivery personnel `(FR-29, FR-30)`
- Order status tracking `(FR-24, BR-14)`

OUT OF SCOPE

- **AI of any kind.** Assumption A-13 places artificial intelligence outside the system. No component in §2.2 implements inference, recommendation or prediction.
- **A platform-employed delivery fleet.** A-04 to A-07 and BR-06 fix the arrangement: the platform never employs, dispatches-as-employer, or equips delivery staff. The Delivery Management component manages records *about* people a restaurant already employs; it does not employ anyone.
- **Restaurant-side physical operation.** Food preparation, transport and cold-chain equipment are physical activities outside the software boundary, shown as external in §2.2 and §5.3.
- **Self-service online registration.** Registration is in person, by restaurant staff. There is no public sign-up interface, by design.

SCOPE CAVEATS CARRIED FORWARD

Thirty-seven questions raised during elicitation remain unanswered. Most do not block the architecture, but several block specific behaviour. This document designs the successful paths and records the blocked behaviour in **§9.4** rather than guessing at it. The three that shape the whole design are:

- **No use case identifiers exist in the SRS.** The specification lists user stories and use cases but does not number them. To satisfy the lab's requirement that behavioural design be traced to use case identifiers, a `UC-xx` register has been derived from the eight documented workflows and the functional requirements. **Derived, not quoted** — the derivation is stated in §6.1 and the full register is Appendix A.
- **Order and delivery failure semantics are undefined.** Rejection, cancellation, failed delivery, late delivery, refund and no-show are all unresolved. This blocks a complete order state machine; §6.6 explains what was designed instead.
- **Compliance is an open dependency, not settled scope.** Compliance §7.11 requires the project to identify regulatory obligations, and the answer is not yet decided. No compliance logic appears in this design.

1.3 Definitions, acronyms and abbreviations

Term	Definition as used in this document
`FR-xx`	Functional requirement, numbered FR-01 to FR-30 in the Software Requirements Specification §6.
`BR-xx`	Business rule, numbered BR-01 to BR-16 in the specification §11.
`UN-xx`	User need, numbered UN-01 to UN-24 in the specification §5.
`BG-xx`	Business goal, numbered BG-01 to BG-09.
`A-xx`	Assumption, numbered A-01 to A-15.
`Q-xx`	Open question raised during elicitation and still unanswered.
`UC-xx`	Use case identifier. **Derived for this SDD** from the specification's workflows and functional requirements; see §6.1 and Appendix A.
`D-xx`	Design decision taken in this document. See §7.3.
`NFR`	Non-functional requirement. The specification names these but does not number them, so they are cited by name and section, never by a minted identifier.
`MVC`	Model–View–Controller. Used inside the Presentation Layer only, for user-facing interfaces.
`CoAD`	Component-Oriented Design. Decomposes a system into components with defined interfaces and explicit composition, rather than into a hierarchy of classes.
`OOD`	Object-Oriented Design. Models the domain as collaborating objects with encapsulated state and responsibility.
`Strategy / Adapter / Observer / State`	Design patterns; each defined at its point of use in §7.1.
`OTP`	One-Time Password. Used in the SMS-based credential verification described in §5.3.
`PROPOSED`	The requirements do not decide this, but the design must take a position to be buildable. The position is stated, justified, and reversible. Used only where a design silence would otherwise block implementation.
`UNRESOLVED`	The requirements do not decide this and the answer is a business decision reserved to the stakeholders. No position is taken. Every occurrence is listed in §9.4 with the open question it depends on.

1.4 References

Document	What this design takes from it
Software Requirements Specification (38 slides)	The authoritative source. All 30 FR, 16 BR, 24 UN, 9 BG, 15 A, the 11-entity data model in §8, the external-system boundary in §2.3 and the out-of-scope list.
Requirements Elicitation record (38 slides)	The four main interactions, the stakeholder list, and the intent behind each user need; used to derive the use case register in Appendix A.
Lab 4 brief, *Design and Methodology*	The seven-step deliverable set, the two mandated comparison tables, the design-quality checklist in Appendix B, and the IEEE 1016 section structure.
IEEE Std 1016–1998	The nine-section document structure and the expected content of each section.
Cross-document consistency report (`CONTRADICTIONS.md`)	Known specification defects. Each one that touches data design is answered in §3.4 or §4.2 rather than silently copied forward.
Business rules §11 and Compliance §7.11 of the specification	BR-01 to BR-16 in full; the compliance question is carried as unresolved in §9.4.

2System Architecture

2.1 Design methodology and architectural style

This section is the Lab 4, Step 1 deliverable, reproduced unchanged in substance. The two comparison tables use the column headings the lab brief mandates.

METHODOLOGIES

The application is designed using a combination of **Structured Design**, **Object-Oriented Design** and **Component-Oriented Design**. Each is applied at a different level of granularity, because the work at each level differs in kind: one decomposes *process*, one models *structure and behaviour*, and one assembles *independently deployable parts with defined interfaces*.

Option Considered	Fits Your Project?	Reason for Choice / Rejection
Structured Design	Yes — selected	Data Flow Diagrams model the **ordering window** well, because the morning pre-order is a time-boxed process with an unambiguous start (deadline not yet passed) and end (order submitted). It is the right tool for showing the deadline gate and the handoff to preparation, but it cannot express the shared, mutable state of a menu item or an order, so it is not used beyond process level.
Object-Oriented Design	Yes — selected	The domain is entity-rich: an order holds items and a payment status, a menu item has availability that changes independently, and a delivery assignment has its own lifecycle. The access rules (BR-08, BR-09, BR-15) are invariants over that state, which encapsulation expresses far better than a process model. OOD is therefore the modelling style for §3 and §4.
Component-Oriented Design	Yes — selected	Six functional areas from the specification (§6: accounts and access, restaurant and menu, ordering, verification, delivery, payment) map almost one-to-one onto components with narrow interfaces. CoAD is what keeps the platform a single deployable unit while letting each area be developed and tested independently — see §2.2 and §2.3.
Microservices	No — deferred	Rejected for the first version, not permanently. The platform has one team, one deployment target and no independent scaling requirement; splitting six components that share one database would add network failure modes and distributed transactions to a system whose hardest requirement is reliability of a single order. Revisit conditions are stated in §2.1.3.

ARCHITECTURAL STYLE

Option Considered	Fits Your Project?	Reason for Choice / Rejection
Layered Architecture	Yes — primary style	The specification's own module structure is already a layering: presentation, application rules, persistence. A layered style makes the access rules enforceable structurally — if a controller cannot reach persistence, BR-09 and BR-15 hold by construction rather than by discipline. Selected as the primary style.
Client–Server	Yes — communication model	Three distinct client roles (customer, restaurant, delivery person) with an administrator beside them, all against a central store. Client–server is adopted as the communication model under the layered style.
MVC	Yes — confined	Adopted *inside the Presentation Layer only*, for user-facing screens. Applying MVC across the whole system would be a category error, because the business rules in §11 are constraints, not view state.
Event-Driven	Yes — selectively	Used where the specification requires fan-out that must not block the caller: order submission reaching the restaurant (FR-15) and order status reaching the customer (FR-24) and notifications. Deliberately *not* used for anything on the payment or delivery-confirmation path, where ordering guarantees matter more than decoupling.

Option Considered	Fits Your Project?	Reason for Choice / Rejection
Microservices	No — deferred	Same reasoning as in the methodology table. Rejected as an architectural style for the first version, revisit conditions in §2.1.3.

2.1.3 Justification

The application is best served by a **layered architecture** because the dominant constraint on this system is a *correctness* constraint, not a throughput one: the platform-wide access rules (BR-09, BR-15) must hold for every operation, and layering makes them enforceable by making a direct route from a screen to the database structurally impossible. A **client–server** model sits underneath because the system genuinely has three separate client populations acting at different times and places against one central record. **MVC** is confined to the presentation layer, where it earns its keep, and **event-driven** mechanisms are used only for the notification fan-outs the specification requires and never on a path where transaction ordering is load-bearing. **Microservices** are deferred: nothing in the current requirements — team size, independent scaling, deployment independence — justifies the operational cost, and the shared-data requirements (BR-14) would turn every order into a distributed transaction. The decomposition into six components is still performed at design time, so adopting services later is a packaging change rather than a redesign.

Methodologies: Structured Design (process), OOD (model), CoAD (assembly). Style: Layered, primary; Client–Server, communication; MVC, presentation only; Event-Driven, selectively. Microservices deferred.

2.1.4 When microservices would become appropriate

The rejection above is conditional, not permanent. Any **one** of the following, should it be answered, would justify revisiting it and re-running this comparison:

- **An independent scaling trigger.** Order submission peaks for roughly ninety minutes before lunch while menu browsing and delivery tracking are steady. If measurement shows the ordering path saturating independently of the rest, Order Management becomes separable.
- **Independent release cadence.** Payment-provider integration changes when a provider changes its API; delivery tracking changes when the business changes a status. If those two cadences begin forcing each other to wait for a release, the boundary is real.
- **A reliability requirement that a single process cannot meet.** If order submission must stay available during a provider outage, the payment adapter must be separable, because it is the only component with an external dependency in the critical path.
- **A second organisation or tenant** requiring code and data isolation by legal or contractual boundary rather than by access rule.
- **A team split.** The single most common and least technical trigger; a component that must be maintained by a different team on a different cadence is a service boundary in practice.

Until one of these is answered, splitting the system would be speculative complexity. The component interfaces in §2.3 are deliberately drawn at the granularity that would allow it, so that the cost of being wrong is low.

2.2 Component diagram

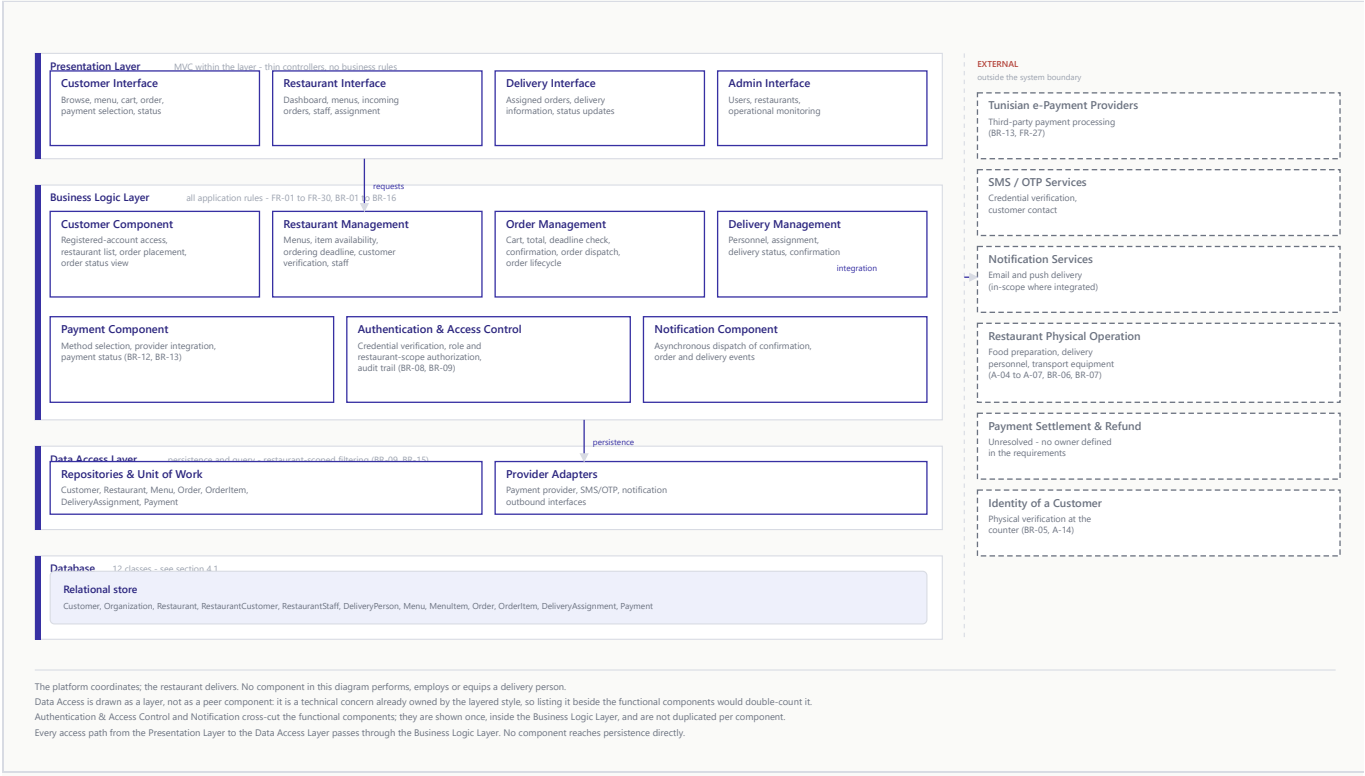


Figure 1 — Component and architecture diagram. `Lab 4, Step 2.` The four layers are drawn as nested bands. Seven components carry the application rules; two carry persistence and outbound integration. Dashed boxes on the right are outside the system boundary. Four reading notes are stated on the diagram itself and are not repeated here.

THREE THINGS THIS DIAGRAM IS DRAWN TO MAKE VISIBLE

1. The platform coordinates; the restaurant delivers. No component performs, employs or equips a delivery person. Delivery Management holds records about people a restaurant already employs. This is the single invariant most likely to be violated by a well-meaning refactor, so it is stated on the diagram and enforced by the class model (§3).

2. There is no direct path to the database. Every access path runs Presentation → Business Logic → Data Access. This is the structural form of BR-09 and BR-15, not a stylistic preference.

3. Data Access is a layer, not a peer component. Listing it beside the seven functional components would double-count it, because it is already a structural layer of the style selected in §2.1. It is shown as its own band for that reason.

2.3 Component descriptions

Component	Layer	Responsibility	Traces to
Presentation Layer — MVC, thin controllers, no business rules			
Customer Interface	Presentation	Browse registered restaurants, view menus, build a cart, submit an order, select a payment method, view order status.	`FR-05, FR-06, FR-09, FR-10, FR-24, FR-25`
Restaurant Interface	Presentation	Incoming order queue, menu and availability editing, ordering-deadline setting, customer verification, delivery personnel and assignment.	`FR-07, FR-08, FR-16, FR-17, FR-18, FR-19`
Delivery Interface	Presentation	Assigned orders, delivery information, status updates, delivery confirmation.	`FR-20, FR-21, FR-22, FR-23`
Admin Interface	Presentation	Platform-level provisioning of restaurant staff accounts; operational monitoring.	`FR-29, UN-24`
Business Logic Layer — all application rules			

Component	Layer	Responsibility	Traces to
Customer Component	Business Logic	Reads and maintains the customer account, the per-restaurant membership record, and the customer's own order history. Enforces that a customer only ever sees restaurants that registered them.	`FR-02, FR-03, FR-04, FR-05, FR-06, FR-09, FR-10, FR-24, BR-04, BR-16`
Restaurant Management	Business Logic	Menus and item availability, the ordering deadline, the customer's organization membership flag, restaurant-owned delivery personnel, and provisioning of those personnel's accounts. Holds every restaurant-scoped rule.	`FR-07, FR-08, FR-16, FR-17, FR-18, FR-30, BR-03, BR-09`
Order Management	Business Logic	The order aggregate and its lifecycle: cart, total calculation, the deadline gate, confirmation, and dispatch to the restaurant. The only component permitted to create an Order.	`FR-11, FR-12, FR-13, FR-14, FR-15, BR-01, BR-02, BR-10, BR-12, BR-14`
Delivery Management	Business Logic	Assignment of an order to a person a restaurant already employs, the delivery information that person needs, delivery status progression, and confirmation of receipt. **Does not employ, dispatch or equip anyone.**	`FR-19, FR-20, FR-21, FR-22, FR-23, BR-06, BR-07, BR-08, BR-11`
Payment Component	Business Logic	Payment-method selection, the provider boundary, and the authoritative payment status. Selects a strategy at runtime; no provider is hard-coded.	`FR-25, FR-26, FR-27, FR-28, BR-12, BR-13`
Authentication & Access Control	Business Logic	Credential verification, role determination, and the restaurant-scope authorisation check that every data access must pass. Emits the audit trail. Cross-cuts the other components; shown once rather than duplicated inside each.	`FR-01, FR-16, FR-29, BR-05, BR-08, BR-09, BR-15`
Notification Component	Business Logic	Asynchronous dispatch of the confirmation, order and delivery events to the parties entitled to receive them. Carries no business decision of its own.	`FR-14, FR-15, FR-24, BR-15`
Data Access Layer — persistence and outbound adapters			
Repositories & Unit of Work	Data Access	Transactional persistence for the twelve classes of §4.1. Applies restaurant-scope filtering as a property of the query, not of the caller, so that BR-09 and BR-15 cannot be bypassed by forgetting a `where` clause.	`BR-09, BR-15`
Provider Adapters	Data Access	Outbound translation to payment providers, SMS/OTP services and notification channels. Isolates every third-party contract behind an adapter so that no provider type reaches a business component.	`FR-27, BR-13, Compliance §7.11`

2.4 Architectural constraints and layer rules

These are the rules the implementation must obey. Each is traceable, and each is testable.

#	Constraint	Why it exists
1	A Presentation component may not contain a business rule. Controllers map input to a component call and output to a view.	Keeps the rules in one place so the access constraints cannot be duplicated and then drift. `BR-09, BR-15`
2	Only the Data Access layer issues queries. A business component never names a table.	Makes the layer boundary real and testable rather than nominal. `BR-09, BR-15`
3	Every persisted entity carries its restaurant identifier, and every repository method requires the caller's restaurant scope as a parameter.	Makes a cross-restaurant leak a type error at the call site rather than a data-leak bug at runtime. `BR-09, BR-15, BR-16`
4	Authentication and authorisation run before the business component, never inside it.	A rule that can be skipped by a call path is not a rule. `BR-08`
5	Only Order Management may create or transition an Order.	BR-14 requires one coherent status; two writers is how the specification's own defect I-1 arises. `BR-14`
6	Third-party contracts are reached only through Provider Adapters.	Provider choice is an unresolved requirement; isolating it means resolving it later costs one adapter. `FR-27, Q-22`
7	No component models, scores or recommends anything.	A-13 places AI outside the system. `A-13`

#	Constraint	Why it exists
8	No component creates, employs, schedules or equips a delivery person.	A-04 to A-07 and BR-06. A restaurant provides its own people. `A-04–A-07, BR-06, BR-07`

3Detailed Design

3.1 Class diagram

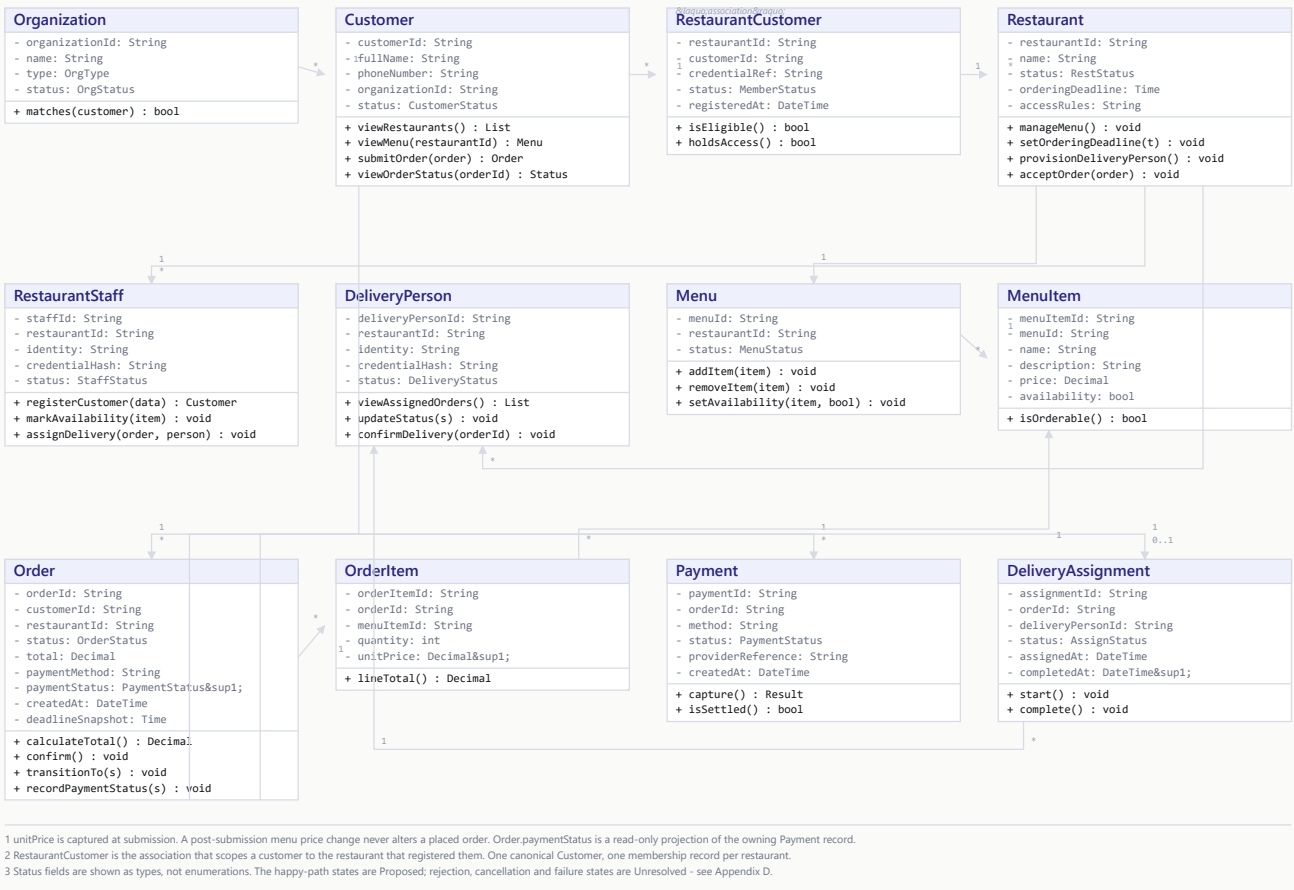


Figure 2 — UML class diagram. `Lab 4, Step 3.` Twelve classes: the eleven entities of the specification's data model plus one association class, `RestaurantCustomer`, without which BR-04, BR-09 and BR-16 cannot be enforced (see D-01). Status attributes are typed rather than enumerated, for the reason given in footnote 3 on the diagram.

> **WHY THE SPECIFICATION'S ELEVEN ENTITIES BECAME TWELVE CLASSES**
>
> The specification's data model has a single `Customer` with no per-restaurant association. That makes FR-01, BR-04, BR-09 and BR-16 unimplementable: nothing in the model can express *which* restaurant registered a given customer, and a single `Customer` row shared by two restaurants would mean one party reading another's order history — a direct breach of BR-15.
>
> Two repairs are possible: give each restaurant its own customer identities, or keep one canonical customer and add a membership record per restaurant. The second is chosen here because it satisfies the eligibility rules without duplicating a person's name and phone number into N rows that could then disagree. The choice is recorded as **D-01** in §3.4 and is `PROPOSED` : it is a design decision, and the specification does not make it.

3.2 Class descriptions

Class	Responsibility	Key state	Traces to
Identity, access and eligibility			

Class	Responsibility	Key state	Traces to
Customer	The person. Holds who they are and how to reach them. Knows nothing about ordering, and no restaurant-specific fact.	`customerId, fullName, phoneNumber, organizationId, status`	`FR-02, FR-03, UN-02`
Organization	The workplace or educational institution a customer belongs to. Needed so a restaurant can decide which customers it serves, which is exactly what FR-17 asks it to record.	`organizationId, name, type, status`	`FR-04, FR-17, UN-03, UN-15`
Restaurant	The party that registers customers, publishes a menu, sets its own ordering deadline, and employs its own delivery personnel. The system's unit of isolation.	`restaurantId, name, status, orderingDeadline, accessRules`	`FR-07, BR-03, BR-04, BR-09`
RestaurantCustomer association	The membership record: *this* restaurant registered *this* customer, they are eligible, and their access for that restaurant. The class that makes BR-04, BR-16 and BR-15 enforceable.	`restaurantId, customerId, credentialRef, status, registeredAt`	`FR-01, FR-17, BR-04, BR-05, BR-15, BR-16`
RestaurantStaff	A member of restaurant staff. Registers customers in person, verifies them, manages the menu, assigns orders.	`staffId, restaurantId, identity, credentialHash, status`	`FR-01, FR-16, FR-17, FR-29`
DeliveryPerson	A delivery person *employed by a restaurant*. The class name is deliberately "Person" and not "Driver": no transport is supplied by the platform (BR-07).	`deliveryPersonId, restaurantId, identity, credentialHash, status`	`FR-18, FR-20, FR-30, BR-06, BR-07`
Menu			
Menu	A restaurant's published menu and its current status. Restaurants define their own menus; the platform does not curate them.	`menuId, restaurantId, status`	`FR-06, FR-07, BR-03`
MenuItem	One orderable item and whether it is currently available. Availability is a property of the item, not of the order, which is what makes BR-02 checkable at submission time.	`menuItemId, menuId, name, description, price, availability`	`FR-08, FR-09, BR-02`
The order aggregate			
Order	The order and its lifecycle. The only aggregate root in the design; the only class allowed to change order status. Owns the total and the deadline snapshot taken at submission.	`orderId, customerId, restaurantId, status, total, paymentMethod, paymentStatus*, createdAt, deadlineSnapshot`	`FR-11, FR-12, FR-13, FR-14, BR-01, BR-12, BR-14`
OrderItem	One line of an order. Captures the unit price at the moment of submission so a later menu change cannot alter a placed order.	`orderItemId, orderId, menuItemId, quantity, unitPrice*`	`FR-09, FR-10, FR-11`
Payment	The authoritative record of money for an order: the method chosen, the provider reference, and the payment status. Sole owner of payment status.	`paymentId, orderId, method, status, providerReference, createdAt`	`FR-25, FR-26, FR-27, FR-28, BR-12, BR-13`

Class	Responsibility	Key state	Traces to
DeliveryAssignment	The link between an order and the person a restaurant sent it with, plus that journey's status. Created before dispatch, per BR-10.	`assignmentId`, `orderId`, `deliveryPersonId`, `status`, `assignedAt`, `completedAt`	`FR-19, FR-22, FR-23, BR-10, BR-11`

* Two attributes are derived rather than owned, and are the subject of design decisions D-02 and D-03 in §3.4: `OrderItem.unitPrice` is a snapshot, and `Order.paymentStatus` is a read-only projection of the owning `Payment`. Neither is independently writable.

3.3 Relationships

Relationship	Multiplicity	Why it is drawn this way
`Organization 1 — * Customer`	1 : 0..*	FR-04 and FR-17 both turn on the customer's institution. Modelling it as a foreign key rather than a derived query keeps FR-17 recordable, which is what the requirement demands.
`Customer 1 — * RestaurantCustomer`	1 : 0..*	A person may be registered at several restaurants. Zero is permitted at the moment of creation because the row is created by the registration, not before it.
`RestaurantCustomer * — 1 Restaurant`	* : 1	The membership is worthless without its restaurant, which is precisely the scoping that BR-16 requires. The pair is unique, so one customer cannot be registered twice at the same restaurant.
`Restaurant 1 — * RestaurantStaff`	1 : 0..*	Staff belong to exactly one restaurant. FR-29 provisions them at platform level; the restaurant relationship is what scopes their authority.
`Restaurant 1 — * DeliveryPerson`	1 : 0..*	BR-06. There is no association to a platform-owned pool, because no such pool exists or may exist.
`Restaurant 1 — 1..* Menu`	1 : 1..*	A restaurant that could publish no menu could not take orders, so at least one menu is required once the restaurant is operating.
`Menu 1 — * MenuItem`	1 : 1..*	An empty menu is not a menu. Items are owned by their menu, never shared between restaurants, which is how BR-03 is respected.
`Customer 1 — * Order`	1 : 0..*	A customer with no orders is normal, so zero is permitted.
`Restaurant 1 — * Order`	1 : 0..*	The second mandatory discriminator on Order. It is the reason no query on Order can be written without a restaurant scope.
`Order 1 — * OrderItem`	1 : 1..*	An order with no lines is not an order. Composition: an order cannot exist without its items, and its items have no meaning apart from it.
`OrderItem * — 1 MenuItem`	* : 1	A line records which item it was. It copies the price rather than reading it back, so that a later price change does not rewrite history (D-02).
`Order 1 — 1..* Payment`	1 : 1..*	BR-12 requires a method on every order, so at least one record must exist at creation. More than one is permitted so a later attempt does not overwrite an earlier one.
`Order 1 — 0..1 DeliveryAssignment`	1 : 0..*	Zero is correct before assignment. The requirement that assignment precede dispatch (BR-10) is a timing rule, not a cardinality, so it is enforced in §6.3 and not here.
`DeliveryAssignment * — 1 DeliveryPerson`	* : 1	Exactly one person per assignment, which is what makes the delivery self-certifying under BR-11 and what scopes the person's access under BR-08.

3.4 Design decisions taken in the class model

Each decision below resolves a place where the requirements are silent but the design cannot proceed without a position. Every one is reversible, and every one is flagged `PROPOSED` because it is the designer's judgement and not a stated requirement.

D-01 ONE CANONICAL CUSTOMER, MEMBERSHIP PER RESTAURANT PROPOSED

Decision. A `Customer` is stored once. Access to any restaurant is held in a separate `RestaurantCustomer` row carrying the restaurant, an eligibility flag and the credential reference.

Why. The alternative — one `Customer` row per restaurant — duplicates name and phone number, and those are already two requirements (FR-02, FR-03). Two restaurants serving the same person would hold two copies of that person's phone number, which can then disagree, and there is no requirement that would let the system reconcile them. The canonical model has one place for identity and one place for scope.

Cost. A restaurant cannot see, by design, that one of its customers is also a customer elsewhere. BR-15 requires exactly this, so it is a feature.

Traces to. FR-01, FR-17, BR-04, BR-09, BR-15, BR-16. **Reverses** the specification's data model in §8, which has no equivalent association.

D-02 UNIT PRICE IS CAPTURED AT SUBMISSION PROPOSED

Decision. `OrderItem.unitPrice` is written once, when the order is submitted, from the menu price at that instant. It is never re-read from the menu afterwards.

Why. BR-14 requires an order status kept throughout the lifecycle, and the specification's own quality clause rules out an incorrect order. A restaurant that raises a price at 11:00 must not retroactively change an order placed at 07:00, nor must a customer be able to add an item to a placed order at yesterday's price.

Traces to. FR-11, BR-14, and the quality clause on a never-inconsistent state. **Answers** the unstated snapshot rule in the specification's `unit_price` field.

D-03 PAYMENT OWNS PAYMENT STATUS; ORDER HOLDS A PROJECTION PROPOSED

Decision. `Payment.status` is the only writable payment status. `Order.paymentStatus` exists only so that a status query does not need a join, and is written by the same transaction from the same value. No code path writes it independently.

Why. The specification carries payment status in two places, which is a contradiction waiting to happen: an order marked paid with no matching payment record is exactly the inconsistency the specification forbids. One writer and one derived copy removes the divergence rather than documenting a rule against it.

Traces to. FR-28, BR-12, BR-14, and the never-inconsistent-state quality clause. **Answers** the two-writable-sources defect in the specification.

D-04 THE DEADLINE IS SNAPSHOTTED ONTO THE ORDER PROPOSED

Decision. `Restaurant.orderingDeadline` is configuration that a restaurant may change. `Order.deadlineSnapshot` is the value that applied to that order, written at submission.

Why. The specification defines the deadline in two places with no stated precedence. The gate on FR-13 must be evaluated against the deadline in force when the customer submitted, not against whatever the deadline has become by the time the order is looked at; otherwise changing a deadline silently changes the legality of orders already placed.

Traces to. FR-12, FR-13, BR-01. **Answers** the duplicated deadline field with no precedence in the specification.

D-05 STATUS FIELDS ARE TYPED, NOT ENUMERATED PROPOSED

Decision. The design fixes the *owner* and the *meaning* of each status attribute, and the happy-path states, but does not publish a closed enumeration. Failure states are not defined at all.

Why. The specification carries ten `status` columns and no enumeration for any of them, and the failure lifecycle is genuinely undecided (Q-04, Q-05, Q-09, Q-20, Q-25). Publishing a closed enumeration now would mean inventing states — `Cancelled`,

`Rejected`, `Failed` — and inventing the *transitions between them*, which is a business decision of exactly the kind reserved to the stakeholders. An open status type with a defined owner and a defined happy path can absorb those states when they are decided; a closed one has to be broken open later.

Cost, stated plainly. The validation layer cannot be completed until the enumeration exists. This is the largest known gap in the design and it is listed in §9.4 rather than glossed over.

Traces to. BR-14, FR-08, FR-22, FR-23, FR-28. **Answers** the unenumerated status columns in the specification.

> D-06 ORGANIZATION IS MODELLED AS A FIRST-CLASS ENTITY PROPOSED

Decision. A `Organization` class exists, referenced by `Customer` and by the membership record.

Why. FR-04 and FR-17 both require the customer's institution to be identifiable and recordable, and the specification's own data model includes it. Earlier drafts of this design omitted the class while keeping the requirement, which left FR-17 unimplementable.

Traces to. FR-04, FR-17, UN-03, UN-15.

3.5 Deliberately absent from the class model

Not modelled	Why it is not here, and what would change if the answer changes
Order lifecycle states — cancelled, rejected, late, failed	Undecided in the requirements. Q-04, Q-05, Q-06, Q-09, Q-20, Q-25. Adding them means deciding who may cancel, whether a restaurant may reject, what happens to the payment, and whether the customer is charged for a failed delivery. All four are business decisions. The *extension point* is designed: `Order.transitionTo()` in the State pattern (§7.1.4) is the single point where a state change occurs, so new states can be added without touching callers.
Refund, void and chargeback	No requirement describes a refund, and the specification does not name who owns settlement. Q-24, Q-25. The `Payment` class has a `capture()` operation and no `refund()`, deliberately.
Delivery fee, price calculation beyond the order total	FR-11 asks only for the total order price. No fee, tax, discount or delivery-charge requirement exists, and Q-31 concerns fees. Nothing is modelled that the requirements do not ask for.
Restaurant menu categories, options, modifiers	FR-07 asks a restaurant to manage its menus; nothing requires categories, sizes, add-ons or variants. Modelling them would be over-design against a checklist that asks for exactly the opposite. A single `MenuItem` with a price is what the requirements support.
Notification, message or email entities	Notifications are not a business record; they are an effect. Persisting them would create a second, unowned copy of order status, which is precisely the defect D-03 removes.
Cart as a persistent entity	No requirement persists an abandoned cart, and no requirement describes recovering one. The cart is a presentation concern that becomes an `Order` at submission.
Any AI, scoring or recommendation entity	A-13. BG-03, order visibility to reduce uncertainty, is met by a status view (§5.2), not by a model.

4Data Design

4.1 Persistent entities

Twelve tables, corresponding one-to-one with the classes of §3.2. Every table carries a surrogate primary key and the restaurant identifier required by constraint 3 in §2.4.

Table	Key	Mandatory columns	Purpose and notes
`organization`	`PK organizationId`	`name, type, status`	Workplace or educational institution. Referenced by `customer`; not scoped to a restaurant, because the same employer can appear at many restaurants. FR-04, FR-17
`customer`	`PK customerId`	`fullName, phoneNumber, FK organizationId, status`	One row per person (D-01). Deliberately carries no `restaurantId`: the person is not owned by a restaurant, their *access* is. FR-02, FR-03
`restaurant`	`PK restaurantId`	`name, status, orderingDeadline, accessRules`	Unit of isolation for every other table. The system boundary owner — not a customer, and not a platform department. BR-09
`restaurant_customer`	`PK (restaurantId, customerId)`	`credentialRef, status, registeredAt`	The association of §3.1. A composite key makes double registration impossible and makes a restaurant-scoped query a prefix match. BR-04, BR-16, FR-01
`restaurant_staff`	`PK staffId`	`FK restaurantId, identity, credentialHash, status`	Accounts provisioned by the platform (FR-29) but scoped to one restaurant. The hash is stored, never the credential. FR-29, BR-05
`delivery_person`	`PK deliveryPersonId`	`FK restaurantId, identity, credentialHash, status`	Accounts provisioned by the restaurant that employs them (FR-30). No transport, equipment or shift fields: the platform does not supply them. FR-06–A-07, BR-07
`menu`	`PK menuId`	`FK restaurantId, status`	At least one per operating restaurant. FR-07, BR-03
`menu_item`	`PK menuItemId`	`FK menuId, name, description, price, availability`	`availability` is the flag BR-02 checks at submission, and it must be read at that moment, not cached. FR-08, BR-02
`orders`	`PK orderId`	`FK customerId, FK restaurantId, status, total, paymentMethod, paymentStatus*, createdAt, deadlineSnapshot`	The aggregate root. Carries both discriminators, which is what forces a restaurant scope on every query. FR-11, FR-12, BR-01, BR-14
`order_item`	`PK orderItemId`	`FK orderId, FK menuItemId, quantity, unitPrice*`	Write-once line (D-02). `unitPrice` is a snapshot and is not a foreign-key follow to the menu. FR-11
`payment`	`PK paymentId`	`FK orderId, method, status, providerReference, createdAt`	Sole owner of payment status (D-03). At least one row per order from creation (BR-12); further rows are later attempts, never an overwrite. FR-28, BR-12
`delivery_assignment`	`PK assignmentId`	`FK orderId, FK deliveryPersonId, status, assignedAt, completedAt`	Exists before dispatch (BR-10). Exactly one person, which is what makes BR-11's self-certification unambiguous. FR-19, BR-10, BR-11

* Derived or snapshot attribute. See D-02, D-03 and D-04.

4.2 Integrity rules

#	Rule	Enforcement and rationale
I-1	Every row in an owned table carries a non-null `restaurantId` matching an existing restaurant.	Database check plus a non-nullable repository parameter. This is the structural form of BR-09: a row that cannot exist without a restaurant cannot leak across one.
I-2	Every order has at least one `order_item`.	Deferred to transaction commit, with the order and its items written in one transaction (§4.4). Enforces the 1 : 1..* of §3.3.
I-3	Every order has at least one `payment` row, created in the same transaction as the order.	BR-12 requires a recorded method for every order. Creating the record with the order is the only way to make the rule true rather than merely intended.
I-4	`orders.total` equals the sum of `quantity × unitPrice` over its items.	Computed in the application and re-checked on write. The stored total is a cache of a derivation that is fully determined by the lines, so it can always be repaired. FR-11
I-5	An `order_item`'s `menuItemId` resolves to an item belonging to the same restaurant as the order.	Checked at submission. A cross-restaurant line would let one restaurant's menu appear in another's order, breaching BR-09 and BR-03 at once.
I-6	An `orders.total` and every `unitPrice` are written once, at submission.	No update path. Enforces D-02 and BR-14, and makes a placed order immutable in value.
I-7	`orders.paymentStatus` equals the status of the order's current `payment` row.	Projection written in the same transaction as the payment (D-03). A divergence is a defect to be raised, not a state to be tolerated.
I-8	A `delivery_assignment` references a `delivery_person` of the same restaurant as the order.	BR-06 and BR-08. A restaurant cannot send its order with a person from another restaurant or from a platform pool, because no such pool may be referenced.
I-9	An order cannot enter a dispatch state without a `delivery_assignment`.	BR-10. The check is in the transition guard, so a future state cannot bypass it.
I-10	No credential is stored in clear text; only a `credentialHash`.	Applies to all three role tables. Required by Compliance §7.11 and by ordinary practice; the compliance obligations themselves remain an open dependency.
I-11	`restaurant_customer` holds the credential reference for that restaurant, not a shared one.	A person registered at two restaurants has two independent memberships and two credentials. This is what "valid only for its creating restaurant" (BR-16) means operationally.

4.3 Derived and snapshot values

Value	Kind	Rule of life
`orders.total`	Derived, cached	Recomputable from the order's items at any time. Not a source of truth; the items are.
`orders.paymentStatus`	Derived, cached	Written only from `payment.status` in the same transaction (D-03). Never independently set.
`order_item.unitPrice`	Snapshot	Written once at submission from the menu price, then immutable (D-02).
`orders.deadlineSnapshot`	Snapshot	Written once at submission from `restaurant.orderingDeadline`, then immutable (D-04).
`order_item.lineTotal`	Derived, not stored	Computed on read as `quantity × unitPrice`.
`Item availability at submission`	Ephemeral check	BR-02 is evaluated against `menu_item.availability` read inside the submission transaction, so a concurrent change by the restaurant is seen rather than raced.

4.4 Transaction boundaries

Three operations must be atomic. The boundary is stated here because it is a data-design decision, and because the failure lifecycle that would add more transactions is unresolved (§6.6).

> **T1 — Order submission** (UC-01, FR-09 to FR-15, BR-01, BR-02, BR-12)
> verify membership · read deadline · read item availability · write order · write items · write payment row · write deadline snapshot → **all or nothing**
> **T2 — Assignment** (UC-02, FR-19, BR-10, BR-06)
> verify person belongs to the order's restaurant · write delivery_assignment → the assignment must exist before anything can be dispatched

T3 — Delivery confirmation (UC-02, FR-23, BR-11)
verify the confirming person owns the assignment · complete assignment · transition order · project payment status if the method requires it → **all or nothing**

Notification dispatch is deliberately **outside** each transaction. It is raised as an event after commit, so a customer is never told an order was placed when it was not, and a notification failure cannot roll back a placed order. This is the reason §2.1 selects event-driven mechanisms selectively rather than wholesale.

5Interface Design

5.1 User interface overview

5.1.1INTERFACE INVENTORY

Interface	Style	Used by	Reached through	Traces to
Customer Interface	Web, mobile-web, responsive	Customer	Customer Component, Order Management, Payment Component	FR-05, FR-06, FR-09, FR-10, FR-24, FR-25
Restaurant Interface	Web, desktop-first	Restaurant staff, restaurant manager	Restaurant Management, Order Management, Delivery Management	FR-07, FR-08, FR-16, FR-17, FR-18, FR-19
Delivery Interface	Web, mobile-web, one-handed	Delivery person	Delivery Management	FR-20, FR-21, FR-22, FR-23
Admin Interface	Web, desktop-first	Platform administrator	Authentication & Access Control, Restaurant Management	FR-29, UN-24
Internal service interface	HTTP/JSON, server-side only	Presentation layer only	The seven business components	see §5.2.1
Provider adapters	Third-party contracts	Payment Component, Notification Component	Data Access layer	FR-27, BR-13

5.1.2CUSTOMER INTERFACE

Four screens, in the order the morning pre-order actually happens. Each names the requirement it implements, so that the interface cannot drift from the specification.

Screen	Purpose and the requirement behind it	Traces to
My restaurants	The list of restaurants that registered this customer, and nothing else. This is the customer-facing half of BR-04: an unregistered restaurant is not merely discouraged, it is not listed and not reachable.	FR-05, BR-04, BR-16
Menu	The selected restaurant's menu with per-item availability shown. Unavailable items are visible but not selectable, because hiding them would make the availability flag invisible to the person who needs to know.	FR-06, FR-08, BR-02
Cart & checkout	Item selection, quantity, the running total, the ordering deadline in plain terms, and payment-method selection. The deadline is shown *before* submission, not enforced only afterwards, so the customer is never refused at the last moment without warning.	FR-09, FR-10, FR-11, FR-12, FR-25, BR-01
Order status	The order's current status and delivery status, updated without the customer refreshing. This is the mechanism that satisfies BG-03: the customer's uncertainty is reduced by *visibility*, not by prediction.	FR-24, BR-14, BG-03, A-13

5.1.3RESTAURANT INTERFACE

Screen	Purpose and the requirement behind it	Traces to
Incoming orders	The order queue for this restaurant only, with the customer information required to verify and fulfil each order. The scoping is applied by the query, not by the screen.	FR-15, FR-16, BR-09, BR-15
Customer register	In-person registration: capture name, phone, workplace or institution, verify identity, and issue the account for this restaurant. There is no public sign-up — registration is only reachable from inside an authenticated staff session.	FR-01, FR-02, FR-03, FR-04, BR-05, BR-16
Organization matching	Records, per restaurant account, whether the customer belongs to this restaurant's target organization. The eligibility flag is a decision this restaurant makes, which is why it is stored on the membership record and not derived globally.	FR-17, BR-04

Screen	Purpose and the requirement behind it	Traces to
Menu editor	Add, edit and remove items, and toggle availability. Item removal is soft: an item that appears on a placed order must still resolve (D-02, integrity rule I-5).	`FR-07, FR-08, BR-03`
Ordering deadline	Sets this restaurant's own deadline. Changing it does not alter the deadline snapshot on orders already placed.	`FR-12, FR-13, BR-01`
Delivery personnel & assignment	Add this restaurant's own delivery personnel, provision their accounts, and assign an order to one of them. The personnel list is necessarily this restaurant's only.	`FR-18, FR-19, FR-30, BR-06`
Payment settings	Enables or disables payment on delivery, per restaurant, since that is a per-restaurant choice in the requirement.	`FR-26`

5.1.4 DELIVERY INTERFACE

Three screens, deliberately minimal. A delivery person is using one hand and is not a professional user, and every extra control is a chance to record a wrong status.

Screen	Purpose and the requirement behind it	Traces to
My orders	Only orders assigned to this person. The filter is not a display preference; the query itself is scoped, so a person cannot reach an unassigned order by changing a parameter.	`FR-20, BR-08`
Order detail	Exactly the information needed to complete that specific delivery — which is the requirement's own wording, and the limit of the data it is entitled to (BR-15).	`FR-21, BR-15`
Delivery status	Update status and confirm delivery. Confirmation is a single deliberate action, because under BR-11 this act is the whole evidence of delivery.	`FR-22, FR-23, BR-11`

5.1.5 ADMIN INTERFACE

Provisions accounts for restaurant staff at platform level, and provides the operational monitoring BG-09 asks for. The specification names this role as UN-23, restaurant-manager monitoring, and UN-24, administration; the two are separated here because their authority differs in kind: a restaurant manager monitors *its own* operations, an administrator acts *across* restaurants. Conflating them would give restaurant managers a cross-restaurant view, which BR-09 forbids.

5.2 API and integration points

5.2.1 INTERNAL SERVICE INTERFACE

The specification does not prescribe an API, so the following is a PROPOSED surface. It is stated at resource-group level rather than endpoint level, because endpoint-level detail is an implementation decision and the design only needs to fix the *boundary* and its authorisation rule. The rule is uniform and is the same rule as integrity rule I-1: **the restaurant scope is a required input to every resource-scoped call, never inferred from the session alone.**

Resource group	Operations	Authorisation rule and role	Components
`/auth`	`signin, verify-otp, signout, session`	Credential check only. Produces a role and a restaurant scope. **No resource scope required.** *Customer, Staff, Delivery, Admin*	Auth & Access Control
`/restaurants`	`list-registered, get-menu`	Customer: only restaurants with an active membership. No restaurant scope accepted from the caller. *Customer*	Customer Component
`/me`	`profile, organization`	Own record only. *Customer*	Customer Component
`/menus`	`read, create, update-item, set-availability`	Writes require a restaurant scope; the scope must match the menu's owner. Reads by a customer are permitted only through `/restaurants/get-menu` after membership is checked. *Customer read, Staff write*	Restaurant Management
`/customers`	`register, verify, set-eligibility`	Requires a restaurant scope; creates a membership for that restaurant only. Not reachable without an authenticated staff session. *Staff*	Restaurant Management
`/orders`	`submit, get, list-own, list-restaurant, list-assigned`	Three disjoint scopes: own orders, own restaurant's orders, own assignments. A caller may never select which of the three they get. *Customer, Staff, Delivery*	Order Management

Resource group	Operations	Authorisation rule and role	Components
`/deliveries`	`list-personnel, provision, assign, start, update-status, confirm`	Personnel management and assignment require a restaurant scope. Delivery actions are authorised against the assignment, so a person can act only on an order assigned to them. *Staff, Delivery*	Delivery Management
`/payments`	`methods, select, status, capture`	Method selection is by the customer for their own order; capture is by staff or the assigned delivery person for that order. Provider calls never leave the adapter. *Customer, Staff, Delivery*	Payment Component
`/admin`	`provision-staff, monitor`	Platform scope, not restaurant scope. The only group reachable without a restaurant identifier. *Admin*	Auth & Access Control

5.2.2 EXTERNAL INTEGRATION POINTS

External system	Direction and purpose	Requirement basis	Status
Tunisian e-payment providers	Outbound. Capture a payment against an order and receive a provider reference and a resulting payment status. Reached only through the adapter; the business layer sees a strategy interface, never a provider type.	`FR-27, BR-13, UN-10`	Provider identity unresolved Q-22
SMS / OTP services	Outbound. Send a one-time password for credential verification and customer contact. Credentials themselves are never carried over SMS in a recoverable form (integrity rule I-10).	`BR-05, Compliance §7.11`	Adapted; provider unnamed
Notification services	Outbound. Deliver the order, delivery and status messages raised after commit (§4.4).	`FR-14, FR-15, FR-24`	In scope where integrated
Restaurant physical operation	Not a software interface. Food preparation, transport and cold-chain equipment are performed by the restaurant with its own people and equipment.	`A-04 to A-07, BR-06, BR-07`	Out of system boundary
Payment settlement & refund	No interface is designed, because the requirements do not describe a refund and do not name who settles funds.	`Q-24, Q-25`	Unresolved

5.3 Authentication and authorisation

No endpoint for either was present in the earlier design, which made FR-01, FR-16, FR-29 and FR-30 unimplementable as written. The following closes that gap with a deliberate, small design.

Role	Credential and verification	Authorised scope
Customer	Holds one membership per registered restaurant, each with its own credential reference (integrity rule I-11). Verification is by the SMS/OTP path, because registration was verified in person and the phone number is the channel the restaurant already used (FR-03, BR-05).	Own record; own orders; menus of restaurants with an active membership. Nothing else (BR-15).
Restaurant Staff	Account provisioned at platform level (FR-29), bound to exactly one restaurant. Verified against that binding on every session.	Own restaurant's orders, menus, customers and delivery personnel (BR-09).
Delivery Person	Account provisioned by the employing restaurant (FR-30), bound to exactly one restaurant and one identity.	Only orders assigned to them (BR-08). The scope is enforced by the query, so it is structural rather than a filter that could be omitted.
Administrator	Platform-level. Provisioned out of band; there is no in-application route to create an administrator, because a system that can grant itself administrative rights cannot be audited.	Cross-restaurant for provisioning and monitoring. Read-only on operational data; the administrator manages accounts, not orders.

Authorisation is a component, not a filter: it runs before the business component and is never inside it, so a call path that forgot it does not exist (constraint 4, §2.4).

5.4 Interfaces not designed

Interface	Why not, and what would trigger designing it
Public self-registration / sign-up	Directly contradicts BR-04 and BR-16: a customer orders only from a restaurant that registered them, so a public sign-up could not create an eligible account. Registration is reachable only from an authenticated staff session.
Customer-facing order cancellation or modification	No requirement asks for it, and the consequences are undecided: what happens to the payment, and whether a restaurant may refuse a late cancellation. Q-04, Q-05, Q-06.
Refund / settlement interface	No requirement describes a refund, and the specification does not name who owns settlement. Q-24, Q-25.
Delivery tracking by GPS or map	No requirement asks for location. The delivery is confirmed by the person performing it (BR-11), and the customer's need is order *status*, not position (FR-24).
Any AI, recommendation or scoring interface	A-13. BG-03 is met by visibility (§5.1.2), not by a model.
Bulk / batch interfaces, data export, reporting warehouse	No requirement asks for analytics, and the specification explicitly rules out a reporting function.

6Behavioral Design

6.1 Sequence diagrams for key use cases

> **HOW THE UC-XX IDENTIFIERS WERE PRODUCED**
>
> The Software Requirements Specification lists use cases and user stories but does not **number** them, and the lab asks for sequence diagrams "linked to use cases." Rather than leave the linkage broken, a `UC-xx` register was derived from the specification's four main interactions and the six functional-requirement modules, giving one identifier per cohesive use case. **This register is derived, not quoted from the SRS** — Appendix A lists all fourteen with the FRs each covers so the derivation can be checked. If the SRS is later given official use case numbers, Appendix A is the only part of this document that needs renumbering.
>
> Three use cases are drawn below. They were chosen because between them they traverse every layer and all seven business components, and because they carry the two hardest business rules: the ordering deadline (BR-01) and delivery self-certification (BR-11).

6.1.1UC-01 — PLACE A MORNING PRE-ORDER

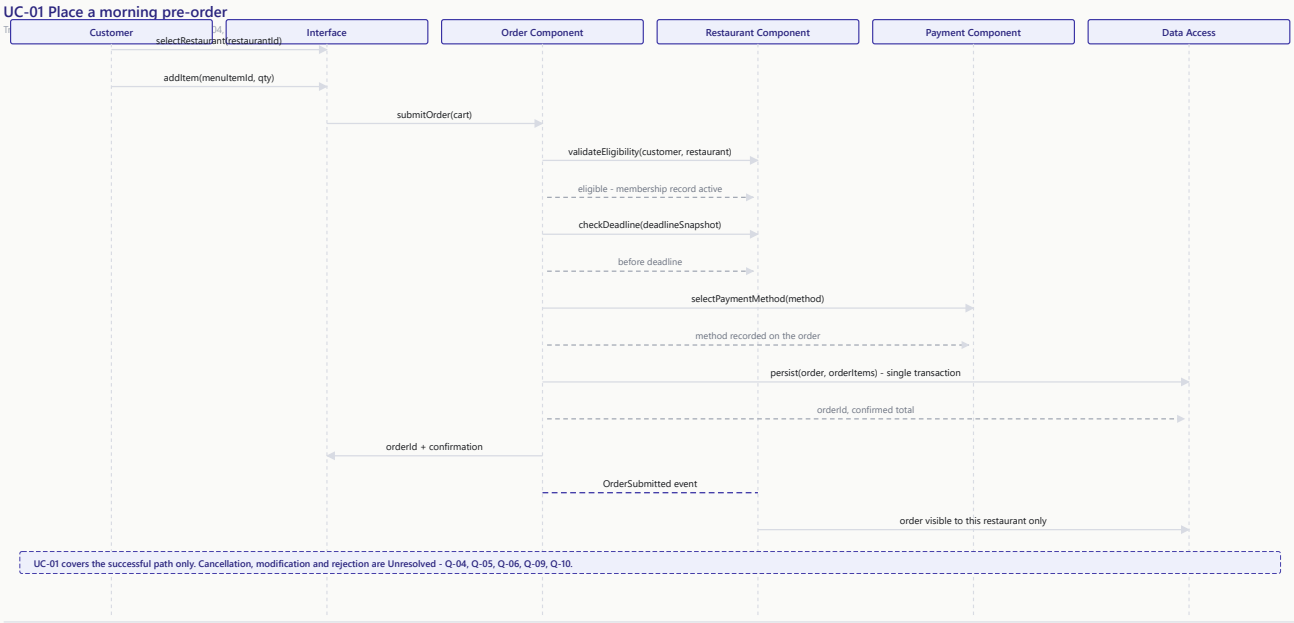


Figure 3 — UC-01, place a morning pre-order. `Lab 4, Step 4.` The submission transaction T1 of §4.4. Note that the order, its items, its payment row and its deadline snapshot are written together, and that the notification is raised *after* commit rather than inside the transaction.

6.1.2UC-02 — ASSIGN A DELIVERY PERSON AND COMPLETE DELIVERY

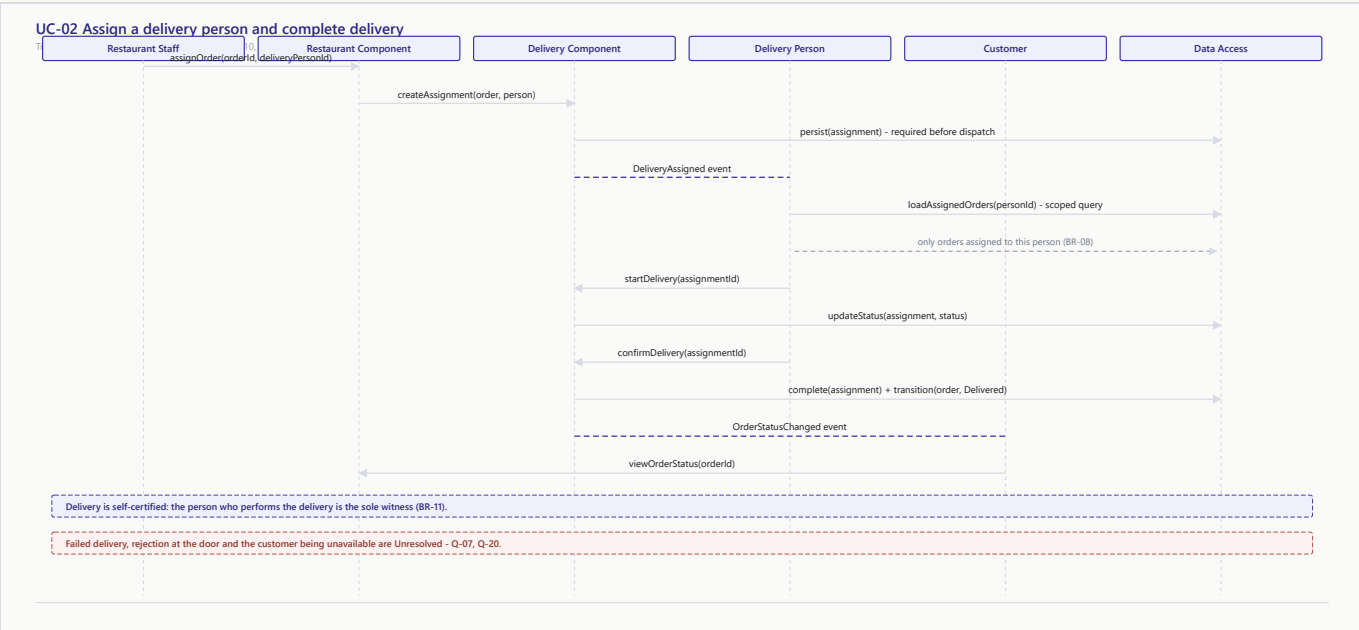


Figure 4 — UC-02, assign a delivery person and complete delivery. `Lab 4, Step 4.` Assignment must be persisted before the order can be dispatched (BR-10), and the assigned person's order list is produced by a scoped query rather than a client-side filter (BR-08).

6.1.3UC-03 — RECORD PAYMENT STATUS ON DELIVERY

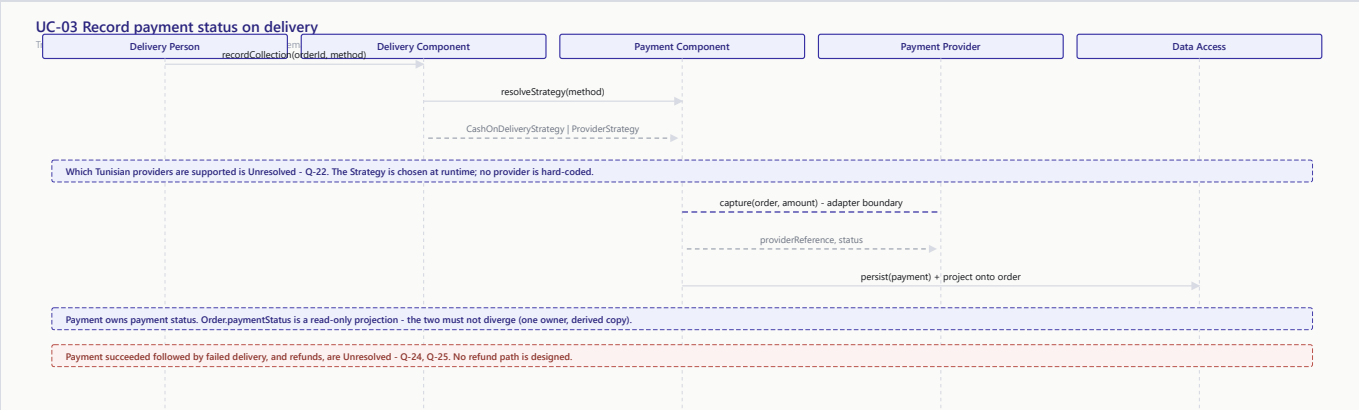


Figure 5 — UC-03, record payment status on delivery. `Lab 4, Step 4.` The strategy is resolved at runtime and the provider is reached only through the adapter (BR-13). Because the provider is unresolved, no provider is named anywhere in this sequence — that is deliberate, not an omission.

6.2 Event catalogue

These are the only events in the design. Each is raised after the transaction that produces it has committed, which is what keeps a notification failure from corrupting a placed order.

#	Event	Raised by → consumed by	Purpose	Traces to
E-01	`OrderSubmitted`	Order Mgmt → Restaurant Mgmt, Notification	Makes the order visible in the correct restaurant's queue and nothing else. Carries the restaurant identifier, so the consumer cannot display it in the wrong place.	`FR-15, BR-09`
E-02	`OrderConfirmed`	Order Mgmt → Customer, Notification	Tells the customer the order exists. Raised after commit, so the customer is never told an order was placed when the transaction rolled back.	`FR-14`
E-03	`DeliveryAssigned`	Delivery Mgmt → Delivery Person, Customer	Notifies the person who must perform the delivery, and the customer that it is on its way.	`FR-19, FR-20, FR-24`
E-04	`DeliveryStatusChanged`	Delivery Mgmt → Customer, Notification	The mechanism that makes FR-24's status view live rather than a page reload.	`FR-22, FR-24, BR-14`

#	Event	Raised by → consumed by	Purpose	Traces to
E-05	PaymentStatusChanged	Payment → Order, Notification	Projects the authoritative payment status onto the order (D-03) and informs the customer. Internal correctness of the projection, not a user-facing feature.	FR-28, BR-12

Events are used only for fan-out. Nothing on the payment-capture or delivery-confirmation path is event-driven, because those two need ordering guarantees more than they need decoupling (§2.1).

6.3 Behaviour deliberately not designed

This is the largest gap in the design and it is stated rather than concealed. The following behaviours are required in spirit by a real system but are **not specified**, and the answers are business decisions that belong to the stakeholders.

Behaviour	Depends on	Why it cannot be designed yet, and the extension point that is ready for it
Order cancellation by customer, before preparation	Q-04, Q-05, Q-06	Nobody has said who may cancel, how late a cancellation is allowed, or what happens to the payment. The extension point exists: `Order.transitionTo()` is the single place a status changes (§7.1.4), so a `Cancelled` state and its guard can be added in one class.
Restaurant rejection of a submitted order	Q-05, Q-09	BR-15 implies the restaurant judges eligibility, but no requirement says a restaurant may refuse an order it has already accepted, nor what the customer is then told. Same extension point.
Failed or unsuccessful delivery	Q-07, Q-20	BR-11 defines confirmation *on success* only. Nobody has said what the person does when the customer is absent, and whether an attempt that failed still counts as an assignment. `DeliveryAssignment.status` is typed and open for it.
Late delivery and deadline breach	Q-08, Q-09	No requirement establishes a delivery-time target or a consequence for missing one. The problem the system addresses is congestion, not speed, so this is a genuine silence rather than an oversight.
Payment succeeded, then delivery failed	Q-24, Q-25	No refund requirement exists and no owner of settlement is named. Designing a refund path would invent both.
Partial order fulfilment or substitution	Q-10, Q-11	No requirement describes an item being unavailable after submission, and BR-02 only guards availability *at* submission.
Concurrent ordering at the deadline boundary	BR-01, FR-12, FR-13	The gate is designed (D-04) and evaluated inside the submission transaction, so the behaviour is well-defined. It is listed here only because the requirement does not say whether a request arriving one millisecond after the deadline is refused; the implementation answers “refused” and this is flagged for confirmation.
Restaurant or customer deactivation	Q-13, Q-14	Every class carries a `status` attribute because the specification includes such a field, but no requirement says what deactivation does to existing orders.

7Design Rationale

7.1 Design patterns used and why

Five patterns, each selected because a specific requirement forced a structure that would otherwise have been written as a conditional. A pattern with no forcing requirement is not included: the specification's own checklist asks that nothing be over-designed, and a pattern applied for style is over-design.

JUSTIFICATION TABLE

Pattern	Problem it solves here	Benefits	Costs and trade-offs	Requirements traced	§
Strategy `PaymentMethodStrategy`	The set of payment methods is per-restaurant configurable *and* open-ended, and no provider is named. Conditional logic in order placement would need editing for every provider, and would couple the two most change-prone areas.	A new provider is an adapter plus a registry entry; the ordering path is untouched. The design is buildable while Q-22 is still open.	One indirection on every payment; a developer reading the ordering path must follow the strategy. Cost grows with each provider, so the alternative is worse over time.	`FR-25, FR-26, FR-27, BR-12, BR-13`	`7.1.1`
Adapter `PaymentAdapter, NotificationAdapter, SmsOtpAdapter`	Three third-party contracts, none specified, none under this team's control, and a compliance section that treats regulatory obligations as an open question.	The business layer is testable against a stub; a provider change is one file; the compliance dependency becomes an isolated replaceable component.	Translation code, and a mapping layer that can lag a provider's model. Also a place to hide business logic, so adapters are kept to protocol translation only.	`FR-27, BR-13, Compliance §7.11, Q-22`	`7.1.2`
Observer `OrderStatusListener`	One status change has to reach three audiences with different rights: customer, restaurant, delivery person. Without decoupling, the order component accumulates knowledge of all three interfaces.	The publisher has no recipient knowledge, so BR-15's recipient rules live in the listener that holds the rights. A new channel is a new listener. FR-24's live status view falls out of the same mechanism.	Hard to follow in a debugger, and a real hazard: a handler that throws must not roll back a committed order. Handled by raising every event after commit and logging handler failures.	`FR-14, FR-15, FR-19, FR-22, FR-24, BR-14, BR-15`	`7.1.3`
State `OrderState` partial state set	BR-14 requires a status kept throughout the lifecycle, and illegal transitions must be impossible — but the failure states are undecided, so no closed enumeration can be published honestly.	Illegal transitions become inexpressible rather than merely discouraged. The happy path is encoded once, and new states can be added as one class with their guards, changing no call site.	More classes for the same behaviour, and a status validation layer that cannot be completed until Q-04 and Q-05 are answered. Stated openly in D-05 and §9.4 rather than closed with invented states.	`BR-14, BR-10, BR-11, FR-22, FR-23, FR-28`	`7.1.4`
MVC `presentation layer only`	Three client roles with almost no screen overlap, over rules that are constraints rather than view state and must hold whichever screen is open.	No path exists from a screen to the database that skips a business component, so the access rules hold structurally rather than by discipline.	Three view sets over one domain model, and a mapping layer between domain and view. Accepted because the roles genuinely differ.	`FR-05–FR-08, FR-16–FR-24, FR-29, BR-09, BR-15`	`7.1.5`

Patterns deliberately *not* applied, and why: **Repository per aggregate only** — rejected, because reading an assigned-order list or a menu would drag the whole order aggregate through a projection query (§7.2). **Singleton** — not applied, because a single global connection manager is a testing liability and the inventory needs no such singleton. **Decorator** — not applied, because the only candidate was delivery-fee composition and no fee requirement exists (Q-31), so a decorator would have nothing to decorate (§3.5).

7.1.1 Strategy — payment method

PROBLEM THE REQUIREMENTS FORCE

FR-25 requires the customer to select *an available* payment method. FR-26 requires payment on delivery *where enabled by the restaurant*. FR-27 requires approved third-party Tunisian electronic payment *where integrated*. The set of methods is therefore both per-restaurant configurable and open-ended — and the requirement names no provider (Q-22).

WHAT WOULD HAPPEN WITHOUT IT

A chain of `if method == ... else if ...` inside order placement, with a provider's specific fields leaking into the order component. Adding a provider would mean editing the ordering logic, retesting the whole ordering path, and coupling the two most change-prone areas together.

SOLUTION

A `PaymentMethodStrategy` interface with one implementation per method family — `CashOnDeliveryStrategy` and `ProviderPaymentStrategy`. The strategy is selected at runtime from the method the customer chose. Every concrete provider sits behind `ProviderPaymentStrategy`, so the number of strategies is small and stable while the number of providers may grow.

CONSEQUENCES

Benefit: a new provider is a new adapter and a registry entry; the ordering path is untouched. It also makes the unresolved Q-22 harmless — the design does not need to know the provider to be buildable.
Cost: one extra indirection on every payment; a developer reading the ordering path must know to follow the strategy. Accepted, because the alternative cost grows with every provider.

TRACES TO

FR-25, FR-26, FR-27, BR-12, BR-13. **Status: decided**

7.1.2 Adapter — external providers and channels

PROBLEM THE REQUIREMENTS FORCE

The system integrates with third-party payment services, SMS/OTP infrastructure and notification services. None of these is specified, and the specification's own compliance section (§7.11) treats regulatory obligations as an open question. Every one of these contracts will change, and none of them is under this team's control.

WHAT WOULD HAPPEN WITHOUT IT

Provider SDK types appearing in business components; compliance changes touching ordering code; the business layer unable to be tested without a live third-party account.

SOLUTION

One narrow interface per external contract, implemented by a Provider Adapter in the Data Access layer (constraint 6, §2.4). The business layer depends on the interface only. The pattern is the natural companion to Strategy here: Strategy chooses *which* capability, Adapter hides *whose* protocol it is.

CONSEQUENCES

Benefit: the whole business layer is testable against a stub; a provider change is one file; the compliance dependency becomes an isolated, replaceable component rather than a property of the system.
Cost: translation code, and a mapping layer that can lag a provider's model. Accepted; the translation is the price of not letting an unknown third party into the core.

TRACES TO

FR-27, BR-13, Compliance §7.11, Q-22. **Status: decided**

7.1.3 Observer — order status propagation

PROBLEM THE REQUIREMENTS FORCE

One status change has to reach three different audiences with different rights: the customer (FR-24, FR-14), the restaurant (FR-15), and the delivery person (FR-19, FR-22). FR-24 is the only Medium-priority requirement in the specification, which suggests the customer is expected to check rather than be pushed to — so the design must make pushing easy without making it mandatory.

WHAT WOULD HAPPEN WITHOUT IT

Every status transition would have to know its recipients and call them, so the order component would accumulate knowledge of notification, of the customer interface, and of the restaurant interface. Adding a fourth listener means editing the order component.

SOLUTION

`Order` publishes the five events of §6.2 to registered listeners. The Notification Component and the three interfaces are listeners. The order component knows only that something is listening.

CONSEQUENCES

Benefit: the order component has no recipient knowledge, so BR-15's recipient rules live in the listener that has the rights, not in the publisher. A new channel is a new listener.
Cost: an indirection that is hard to follow in a debugger, and a real hazard — an event handler that throws must not roll back a committed order. That is why every event is raised after commit (§4.4) and why handler failures are logged rather than propagated.

TRACES TO

FR-14, FR-15, FR-19, FR-22, FR-24, BR-14, BR-15. Status: decided

7.1.4 State — order lifecycle

PROBLEM THE REQUIREMENTS FORCE

BR-14 requires an order status kept throughout the lifecycle, FR-22 and FR-23 move a delivery through states, FR-28 records a payment status, and the specification's quality clause forbids a state that ever contradicts itself. Something must prevent illegal transitions — but the failure states are undecided (§6.3).

WHAT WOULD HAPPEN WITHOUT IT

Status checks scattered as `if` conditions on every path that touches an order. That is exactly the arrangement that produces a status which contradicts itself, because the rule is only enforced on the paths someone remembered to write it on.

SOLUTION, AND ITS HONEST LIMIT

An `OrderState` hierarchy whose states own the transitions they permit. **The pattern is selected; the state set is deliberately incomplete.** The happy path is designed — placed, confirmed, preparing, assigned, out for delivery, delivered, completed — because the specification's own end-to-end scenario walks it. The failure states are absent, because choosing them is a business decision (Q-04, Q-05, Q-09, Q-20, Q-25) and D-05 forbids inventing them.
This is the pattern's purpose as much as its limitation: the happy path is encoded once, and adding `Cancelled` or `Rejected` later means adding a class and the guards it needs — no call site changes. A design that pretended to know the failure states would be more complete-looking and less safe, because the invented transitions would then be coded and depended upon.

CONSEQUENCES

Benefit: illegal transitions become impossible to express rather than merely discouraged; the incomplete state set is honest and extensible.
Cost: more classes for the same behaviour, and a status validation layer that cannot be completed until Q-04 and Q-05 are answered. That cost is stated in D-05 and in §9.4 rather than hidden.

TRACES TO

BR-14, BR-10, BR-11, FR-22, FR-23, FR-28. Status: pattern decided, state set deliberately partial

7.1.5 MVC — presentation layer only

PROBLEM THE REQUIREMENTS FORCE

Three client roles, each with a distinct screen set, over rules that are constraints rather than view state. The eligibility rule, the deadline gate and the assignment scope all have to hold no matter which screen is open.

SCOPE, DELIBERATELY NARROW

MVC is applied inside the Presentation Layer only. Applying it system-wide would place business constraints in a model layer that also serves the view — and a rule that the view can override is not a rule. This confinement is a decision recorded in §2.1, and it is why the layered style is the primary architecture and MVC is not.

SOLUTION

Thin controllers per role, views per screen, and no business object in the presentation layer. All rules live in the seven business components, reached through the internal service interface of §5.2.1.

CONSEQUENCES

Benefit: the access rules hold for every path, because no path exists from a screen to the database that does not pass through a business component (constraint 2, §2.4).
Cost: three view sets to maintain for one domain model, and an extra mapping layer between domain and view. Accepted, because the roles genuinely differ; a customer, a restaurant and a delivery person share almost no screen.

TRACES TO

FR-05 to FR-08, FR-16 to FR-24, FR-29, BR-09, BR-15. Status: decided

7.2 Alternatives considered and trade-offs

Alternative	Verdict	Trade-off
Microservices from the start	Deferred	*For:* independent deployment, independent scaling of the ordering peak, fault isolation from payment providers. *Against:* BR-14's single coherent order status would require distributed transactions; one team and one deployment target make the operational cost unpaid; the shared-database queries that BR-09 mandates are a network hop, not an index. Revisit triggers in §2.1.4.
Event sourcing for order status	Rejected	*For:* a status history is the audit trail BR-15 needs, for free and with perfect reconstruction. *Against:* the specification never asks for history, only for a current status; read models then have to be built and kept correct; and it would make the unresolved failure lifecycle harder, not easier, because every speculative event type would need a schema. The audit requirement is met by ordinary timestamps.
Repository per aggregate only	Rejected	*For:* fewer boundaries, stronger encapsulation of the order. *Against:* reading a delivery person's assigned order list would drag the whole order aggregate in through a delivery-side query, and menu browsing would too. Read models are legitimate where the requirement is a projection, not an edit.
A single Customer entity with per-restaurant copies	Rejected	*For:* no association class needed; isolation is total. *Against:* FR-02 and FR-03 make the name and phone number requirements, and N copies of a person's phone number can disagree with no requirement to reconcile them. Chosen against in D-01.
Business rules in the database (triggers, stored procedures)	Rejected	*For:* the rules cannot be bypassed by any client. *Against:* BR-01, BR-02 and BR-05 need values from the current request, not from the row; testing them requires a database instead of a unit test, which the checklist in Appendix B explicitly asks about. The structural protection is achieved instead by constraint 3 of §2.4, which is testable without a database.
Presenting order status by polling rather than pushing	Not selected	*For:* simpler, no event infrastructure, nothing to fail at the worst moment. *Against:* FR-24 asks customers to view status, and polling costs a request per customer per interval during exactly the ninety-minute peak the system exists to handle. The event is raised after commit, so its failure cannot corrupt an order; polling is the right fallback, and the client should degrade to it.
GraphQL instead of the resource interface	Rejected	*For:* three role-specific screens could each fetch exactly what they need. *Against:* the authorisation rules in §5.2.1 are clearer and more auditable when the server decides what a role may read. A client-chosen query shape makes BR-09 and BR-15 dependent on every client's correctness.
Modular monolith with per-module schemas	Deferred	*For:* keeps the option of extraction without network calls today. *Against:* BR-09 and BR-15 need restaurant-scoped joins across modules today, and a single schema serves them better. Worth revisiting only with the same triggers as microservices (§2.1.4).

7.3 Design decision register

ID	Decision	Alternative rejected	Reason and requirement basis	Status
D-01	One canonical Customer; membership per restaurant	Per-restaurant customer identities	FR-02, FR-03 make identity requirements; duplicated identities can diverge. Makes BR-04, BR-09, BR-15, BR-16 enforceable.	PROPOSED
D-02	Unit price snapshotted at submission	Read price from the menu on demand	A price change would retroactively alter a placed order, breaching BR-14 and the never-inconsistent-state quality clause. FR-11.	PROPOSED
D-03	Payment owns payment status; Order projects it	Two writable sources of payment status	The specification carries payment status twice; two writers is how an order becomes paid with no payment record. FR-28, BR-12, BR-14.	PROPOSED
D-04	Ordering deadline snapshotted onto the order	Single live deadline field	The specification defines the deadline twice with no precedence. Changing a deadline would change the legality of orders already placed. FR-12, FR-13, BR-01.	PROPOSED
D-05	Status fields typed, not enumerated	Closed enumeration of all states	The failure lifecycle is undecided; a closed set means inventing states and their transitions. Open types absorb them later. BR-14.	PROPOSED
D-06	Organization is a first-class class	Workplace and institution as free text on Customer	FR-17 requires the membership to be recordable and FR-04 requires the institution to be identifiable. Free text cannot support either.	PROPOSED
D-07	Data Access drawn as a layer, not a peer component	List persistence as an eighth functional component	Persistence is a structural layer of the chosen style; listing it as a component double-counts it and weakens the component boundary.	Structural
D-08	Notifications raised after commit, never inside a transaction	Dispatch inside the order transaction	An order must never be rolled back because a notification failed, and a customer must never be told an order was placed when it was not. FR-14, FR-15, FR-24.	Structural
D-09	Status transitions funnel through one method	Direct status writes from each caller	BR-10 and BR-11 are transition guards; scattered checks are enforced only on the paths someone remembered. BR-14.	Structural
D-10	Restaurant scope is a required repository parameter	Scope inferred from the session inside the repository	Making it a parameter means a caller that forgets it fails to compile, so BR-09 and BR-15 cannot be bypassed by an omission.	Structural
D-11	UC-xx identifiers derived for this SDD	Cite use cases without identifiers	The lab requires sequence diagrams linked to use cases; the SRS numbers none. A derived register, with its derivation shown in Appendix A, is better than a broken link.	PROPOSED
D-12	No cancellation, rejection or refund behaviour designed	Design a plausible failure path	Deciding who may cancel, whether a restaurant may refuse, and who refunds are business decisions the requirements do not make. Extension points are in place.	UNRESOLVED

8Requirements Traceability Matrix

Column headings are those the lab brief prescribes. Every functional requirement appears exactly once, with the design element that implements it and a note carrying its originating user need, its use case and any business rule it rests on.

8.1 Functional requirements to design elements

Requirement ID	Design Element (Component/Class)	Notes
FR-01	Auth & Access Control; Restaurant Management — RestaurantCustomer, RestaurantStaff.registerCustomer()	UN-01. UC-04. Creates the membership record that carries the restaurant scope. BR-04, BR-05, BR-16. Reachable only from an authenticated staff session.
FR-02	Customer Component — Customer.fullName	UN-02. UC-04. Integrity rule I-10 applies to the related credential.
FR-03	Customer Component — Customer.phoneNumber	UN-02. UC-04. Also the OTP delivery channel in §5.3.
FR-04	Customer Component; Restaurant Management — Customer.organizationId, Organization	UN-03. UC-04, UC-08. D-06. Without the class, FR-04 is not implementable.
FR-05	Customer Component — RestaurantCustomer query	UN-04. UC-05. BR-04, BR-16. The query is driven by active memberships, so an unregistered restaurant cannot be listed.
FR-06	Customer Component — Menu, MenuItem	UN-05. UC-06. BR-03, BR-09. Reached only through an active membership.
FR-07	Restaurant Management — Menu, MenuItem	UN-16. UC-07. BR-03, BR-09. Item removal is soft so placed orders still resolve (D-02, I-5).
FR-08	Restaurant Management — MenuItem.availability	UN-16. UC-07. BR-02. Read inside the submission transaction, not cached (§4.3).
FR-09	Customer Component; Order Management — OrderItem.menuItemId, MenuItem	UN-06. UC-01. BR-02.
FR-10	Customer Component; Order Management — OrderItem.quantity	UN-07. UC-01.
FR-11	Order Management — Order.calculateTotal()	UN-07. UC-01. Integrity rule I-4. D-02: computed from the snapshotted line prices, not from live menu prices.
FR-12	Order Management — Order.deadlineSnapshot	UN-06. UC-01. BR-01. D-04. Evaluated inside transaction T1.
FR-13	Order Management — transition guard on Order.transitionTo()	UN-06. UC-01. BR-01. D-09. Refusal behaviour at the exact boundary is flagged in §6.3.
FR-14	Order Management; Notification Component — OrderConfirmed (E-02)	UN-06. UC-01. D-08: raised after commit.
FR-15	Notification Component — OrderSubmitted (E-01)	UN-12. UC-01. BR-09: the event carries the restaurant identifier, so it can only land in the right queue.
FR-16	Restaurant Management — RestaurantCustomer, Customer	UN-14. UC-08. BR-15: the minimum information required to verify and fulfil, per the requirement's own wording.
FR-17	Restaurant Management — RestaurantCustomer.status, Organization	UN-15. UC-08. D-06. The eligibility flag is per restaurant account, which is why it lives on the membership.
FR-18	Restaurant Management; Delivery Management — DeliveryPerson	UN-17, UN-18. UC-09. BR-06, BR-07, BR-09. The list is necessarily this restaurant's own personnel only.
FR-19	Delivery Management — DeliveryAssignment.createAssignment()	UN-17. UC-02. BR-10. The assignment is persisted before dispatch (transaction T2).
FR-20	Delivery Management — scoped assignment query	UN-20. UC-10. BR-08. Enforced by the query, not by a client-side filter.

Requirement ID	Design Element (Component/Class)	Notes
FR-21	Delivery Management — Order, RestaurantCustomer projection	UN-21. UC-10. BR-15: only the information needed for that specific delivery is exposed.
FR-22	Delivery Management — DeliveryAssignment.status	UN-22. UC-11. D-05: typed, not enumerated.
FR-23	Delivery Management — DeliveryAssignment.complete(), Order.transitionTo()	UN-22. UC-02. BR-11. Transaction T3. The person performing the delivery is the sole witness, by requirement.
FR-24	Customer Component; Notification Component — DeliveryStatusChanged (E-04)	UN-11. UC-12. BR-14. The only Medium-priority FR; the client is designed to degrade to polling if the event does not arrive.
FR-25	Payment Component — Payment.method, PaymentMethodStrategy	UN-09, UN-10. UC-13. BR-12. §7.1.1.
FR-26	Payment Component — CashOnDeliveryStrategy	UN-09. UC-13, UC-03. Enabled per restaurant, as the requirement states.
FR-27	Payment Component; Provider Adapters — ProviderPaymentStrategy, PaymentAdapter	UN-10. UC-03. BR-13. §7.1.1, §7.1.2. Provider identity is unresolved (Q-22) and no provider is named.
FR-28	Payment Component — Payment.status	UN-09, UN-10. UC-03. BR-12. D-03: sole writable payment status; the order's copy is a projection. Integrity rule I-7.
FR-29	Auth & Access Control — RestaurantStaff	UN-24. UC-14. Platform scope; the account is still bound to exactly one restaurant. §5.3.
FR-30	Restaurant Management — DeliveryPerson	UN-18. UC-14. BR-06. Provisioned by the employing restaurant, not by the platform.

8.2 Business rules to enforcement points

Requirement ID	Design Element (Component/Class)	Notes
BR-01	Order Management — deadline guard in T1; Order.deadlineSnapshot	FR-12, FR-13. D-04. The guard reads the snapshot, so a later deadline change does not affect an order already placed.
BR-02	Restaurant Management — MenuItem.availability read in T1	FR-08, FR-09. Read inside the transaction so a concurrent restaurant change is seen rather than raced.
BR-03	Restaurant Management — Menu ownership	FR-07. A menu belongs to exactly one restaurant; integrity rule I-1 makes a cross-restaurant item unreachable.
BR-04	Customer Component — RestaurantCustomer.isEligible()	FR-05, FR-16. D-01. This rule is unimplementable without the membership class.
BR-05	Auth & Access Control — registration workflow	FR-01. Verification is in person; the software records its outcome, and the OTP path in §5.3 continues the same channel.
BR-06	Delivery Management — DeliveryPerson.restaurantId; integrity rule I-8	FR-18, FR-19, FR-30. The database cannot hold a reference to a platform delivery pool, because no such entity exists and none may be created (A-04 to A-07).
BR-07	Delivery Management — no equipment or transport entity	No requirement asks for equipment management; modelling it would be over-design. Stated as an absence on purpose, per §3.5.
BR-08	Delivery Management — scoped query; §5.3 role scope	FR-20. D-10. Scoping is a required query parameter, so omitting it does not compile.
BR-09	All repositories; integrity rule I-1; constraint 3, §2.4	Every owned table carries a restaurant identifier. FR-05 to FR-08, FR-15 to FR-19.
BR-10	Order Management — transition guard; integrity rule I-9	FR-19. D-09. The guard is in the transition, so a future state cannot bypass it.
BR-11	Delivery Management — DeliveryAssignment (exactly one person); transaction T3	FR-23. One person per assignment is what makes the confirmation unambiguous.

Requirement ID	Design Element (Component/Class)	Notes
BR-12	Payment Component — Payment row created with the order; integrity rule I-3	FR-25, FR-28. The rule is made true by construction rather than by a check that could fail.
BR-13	Provider Adapters — PaymentAdapter registry	FR-27. Only providers in the registry are reachable; there is no generic “call any provider” path.
BR-14	Order — OrderStatus via the State pattern; D-05	FR-22, FR-23, FR-24. One writer, transitions guarded in one place. The state set is deliberately partial (§6.3).
BR-15	Auth & Access Control; Observer listeners carry recipient rights	FR-16, FR-21, FR-24. §5.2.1, §5.3. Each listener is entitled to what it receives, so the rule is expressed once per audience rather than once per sender.
BR-16	Customer Component — per-restaurant membership and credential; integrity rule I-11	FR-01, FR-05. D-01. Two restaurants serving one person hold two independent memberships and two credentials.

8.3 Coverage summary

Measure	Count	Result
Functional requirements in the specification	30	—
Functional requirements mapped to at least one design element	30	Complete §8.1
Functional requirements with an originating user need identified	30	Complete UN links restored in §8.1; they had been dropped from the requirement set.
Business rules in the specification	16	—
Business rules mapped to an enforcement point	16	Complete §8.2. BR-07 is mapped to a deliberate absence; the note says so.
Design elements in the class model	12	Eleven specification entities plus the `RestaurantCustomer` association required by D-01.
Components carrying application rules	7	Plus two Data Access components. See §2.3.
Use cases modelled behaviourally	3	Lab minimum is two. Chosen to traverse all layers; the other eleven are in Appendix A.
Design patterns applied	5	Lab minimum is two.
Open questions blocking a specific design element	8 behaviours in §6.3	Open — listed in §9.4, not designed.
Design decisions taken where the requirements are silent	6 PROPOSED	§3.4 and §7.3. Each is justified and reversible.

No functional or business requirement is left without a design element, and no design element is left without a requirement behind it, except the twelve items deliberately excluded in §3.5, each of which is recorded with the reason it was excluded.

9Appendices

A Use case register

Derived for this SDD (D-11). The specification lists use cases without numbers; this register gives one identifier per cohesive use case so that behavioural design can be traced. Derivation rule: the four main interactions and the six functional-requirement modules of the specification §6 were partitioned into use cases that each have a single initiating actor, a single outcome and a single business purpose.

UC	Use case	Initiating actor	Functional requirements	Modelled in
UC-01	Place a morning pre-order	Customer	FR-09 – FR-15	**§6.1.1**, Figure 3
UC-02	Assign a delivery person and complete delivery	Restaurant staff, then delivery person	FR-19 – FR-23	**§6.1.2**, Figure 4
UC-03	Record payment status on delivery	Delivery person	FR-25 – FR-28	**§6.1.3**, Figure 5
UC-04	Register a customer in person	Restaurant staff	FR-01 – FR-04	§5.1.3, §5.3
UC-05	View the restaurants that registered me	Customer	FR-05	§5.1.2
UC-06	Browse a restaurant menu	Customer	FR-06	§5.1.2
UC-07	Manage menus and item availability	Restaurant staff	FR-07, FR-08	§5.1.3
UC-08	Verify a customer and record organization membership	Restaurant staff	FR-16, FR-17	§5.1.3
UC-09	Manage the restaurant's delivery personnel	Restaurant staff	FR-18	§5.1.3
UC-10	View assigned orders and delivery information	Delivery person	FR-20, FR-21	§5.1.4
UC-11	Update delivery status	Delivery person	FR-22	§5.1.4
UC-12	View order and delivery status	Customer	FR-24	§5.1.2, §6.2
UC-13	Select a payment method	Customer	FR-25, FR-26	§5.1.2, §7.1.1
UC-14	Provision staff and delivery accounts	Administrator, restaurant staff	FR-29, FR-30	§5.3

B Design quality checklist

The seven items are those of Lab 4, Step 6.1, stated as written in the brief. Each is answered honestly: a partial pass is recorded as a partial pass, and the reason is given.

Checklist item	Verdict	Basis, and what is still open
Every functional requirement is addressed by at least one design element	Pass	All 30 mapped in §8.1, and all 16 business rules in §8.2. The user-need links dropped from the requirement set were restored, so traceability now runs requirement → need → requirement → component → class → use case.
Components/classes have high cohesion (a single, clear responsibility)	Pass	Each of the seven business components owns one specification module. One deliberate exception is recorded: Authentication & Access Control is cross-cutting and shown once rather than duplicated, and its two responsibilities — verification and scope — are not separable without repeating it in every component.
Coupling between components is minimized (clear interfaces, no leaked internals)	Pass	Components exchange only component operations; no class of one component is named by another. Two deliberate couplings remain and are stated: Order Management knows OrderItem (composition), and Payment is projected onto Order (D-03), which is a coupling created deliberately to prevent a correctness bug.

Checklist item	Verdict	Basis, and what is still open
The design follows SOLID principles where applicable	Pass	Single responsibility: one module per component. Open/closed: Strategy and Adapter (§7.1.1, §7.1.2) are the mechanism by which a provider is added without editing callers. Liskov, interface segregation and dependency inversion are satisfied by the repository-per-aggregate pattern and the strategy interface; the narrow interfaces in §5.2.1 are segregated by role rather than by entity.
No requirement is over- or under-designed	Pass	§3.5 lists twelve elements deliberately *not* modelled, each with the requirement that does not ask for it: menu categories, fees, refunds, GPS tracking, AI, reporting, persistent carts, notifications as entities. Under-design is the live risk, not over-design, and is recorded honestly below.
The design supports testability (components can be tested in isolation)	Pass	Every third-party contract sits behind an Adapter (§7.1.2), so no business component needs a live provider to be tested. Business rules are in the business components, not in triggers or stored procedures, so they can be unit-tested without a database. Notification dispatch is after commit, so a component's behaviour does not depend on a message bus being available.
Non-functional requirements (performance, security, scalability) are reflected in the design	Partial	**Security** is reflected: authorisation is a component that runs before the business layer (constraint 4), credentials are hashed (I-10), scopes are enforced by required query parameters (D-10), and the administrator has read-only access to operational data. **Performance and scalability** are reflected only indirectly — the design keeps an event fan-out off the ordering path, and the data model's composite key on `restaurant_customer` makes the hottest restaurant-scoped query a prefix match. **No explicit performance target exists in the specification**, so no capacity figure is designed against; the ninety-minute lunch peak is a known load shape with no stated target. Compliance §7.11 is an open dependency, not a settled requirement. This item cannot honestly be marked as a full pass until the specification supplies a target.

C Self-review log

Lab 4, Step 6.3 asks for a peer review exchanged with a classmate, or a self-review after a break. **No classmate review has taken place, so this is a self-review and is labelled as one.** It is not presented as a peer review. Six issues were found in this design while drafting it; each was fixed, and the fix is verifiable in the section cited.

#	Issue found	Severity	How it was fixed, and where to check
1	FR-04 and FR-17 require the customer's workplace or institution, but no `Organization` class existed, so both requirements had nothing to implement.	Blocking	Added the class and the association, with the eligibility flag on the membership record so it is recorded per restaurant as FR-17 requires. D-06, §3.2, §3.4, Figure 2.
2	BR-04, BR-09 and BR-16 cannot be enforced with the specification's single `Customer` entity, because nothing records which restaurant registered a customer. One shared row would also let two restaurants see each other's orders, breaching BR-15.	Blocking	Introduced the `RestaurantCustomer` association class and integrity rules I-1, I-3, I-11. D-01, §3.1, §3.3, §4.2, Figure 2.
3	Payment status existed in two places, `Order` and `Payment`, both writable. An order could be marked paid with no matching payment record — precisely the inconsistent state the specification forbids.	Blocking	Made `Payment` the sole owner and the order's copy a projection written in the same transaction. D-03, integrity rule I-7, §4.3, §8.1 FR-28.
4	The ordering deadline was defined on both `Restaurant` and `Order` with no stated precedence, so a restaurant changing its deadline would silently change whether an already-placed order was legal.	High	Separated configuration from snapshot: the guard reads `Order.deadlineSnapshot`, written at submission. D-04, §4.3, §8.2 BR-01.

#	Issue found	Severity	How it was fixed, and where to check
5	No authentication or authorisation interface existed, which left FR-01, FR-16, FR-29 and FR-30 unimplementable as written, and made the BR-09 and BR-08 scoping rules unenforceable outside a controller.	High	Added §5.2.1 with the scope rules per resource group, and §5.3 with a credential and scope per role. Also corrected the earlier conflation of restaurant-manager monitoring with platform administration, which would have given managers a cross-restaurant view that BR-09 forbids.
6	The user-need links were absent from the functional requirement set, so traceability stopped at the requirement with no route back to the need it served.	Medium	Restored the `UN-xx` column for all 30 requirements in §8.1, including the three whose link was ambiguous (FR-18, FR-25, FR-28), and added business-rule references. Full chain now runs need → FR → component → class → use case.

STILL OPEN AFTER THIS SELF-REVIEW

Issues 1 to 6 were defects *in the design*, and are fixed. One class of problem was found that is not a design defect and cannot be fixed by design: the unresolved failure lifecycle. Eight behaviours in §6.3 remain undesigned, and no amount of additional design will close them without the stakeholder answers recorded in §9.4. The extension points that will carry the answers are in place, so those decisions can be absorbed when they arrive.

D Unresolved design items

Every item here is a question the requirements do not answer. No design position has been taken on any of them. This register exists so that the design's silence is auditable rather than mistaken for an oversight.

#	Question	Open question	What is blocked, and the extension point ready for it
U-1	May a customer cancel a submitted order, and until when?	`Q-04, Q-05, Q-06`	A `Cancelled` state and its guards. `Order.transitionTo()` is the only place a status changes (§7.1.4), and no refund path exists, so cancellation after payment is also blocked on U-5.
U-2	May a restaurant reject an order it has already received?	`Q-05, Q-09`	A `Rejected` state, and the customer-facing message that goes with it. Same extension point.
U-3	What does a delivery person do if the customer is absent?	`Q-07, Q-20`	A failed-attempt state on `DeliveryAssignment.status`, which is typed and open (D-05). BR-11 defines success only.
U-4	Is there a delivery-time target, and what follows from missing it?	`Q-08, Q-09`	Nothing in the design. No state, no alert, no escalation is invented, because the system addresses congestion rather than speed.
U-5	Who refunds, and when?	`Q-24, Q-25`	A `refund()` operation on `Payment`, which is deliberately absent. Also undecided: who settles funds at all.
U-6	What happens if an item becomes unavailable after submission?	`Q-10, Q-11`	Partial-fulfilment behaviour. BR-02 guards availability only at the moment of submission, and D-02 makes the line price immutable, so there is no mechanism to reprice an order.
U-7	Which Tunisian payment providers are supported?	`Q-22`	Nothing is blocked: Strategy and Adapter (§7.1.1, §7.1.2) were chosen precisely so the design does not need to know. A provider is one adapter and one registry entry.
U-8	What is the boundary behaviour for a request arriving exactly at the deadline?	`FR-12, FR-13, BR-01`	The implementation refuses a request after the deadline; the requirement does not state this. The decision is flagged in §6.3 for confirmation rather than assumed correct.
U-9	What are the Tunisian regulatory obligations, and do they change the design?	`Compliance §7.11`	Potentially data design — retention, consent, localisation of customer data. I-10 (hashed credentials) and the Adapter boundary are the two provisions already made in anticipation. No compliance logic is designed.
U-10	What does deactivating a restaurant or customer do to orders already placed?	`Q-13, Q-14`	Every class has a `status` because the specification includes such a field, but no requirement defines its effect on in-flight orders.
U-11	What are the actual non-functional targets — response time, concurrent users, order volume?	`Specification, unnumbered`	Appendix B item 7 cannot be marked a full pass. The design is deliberately built so that a target can be added without redesign: the lunch peak is handled by keeping fan-out off the ordering path and scoping every query by restaurant.

#	Question	Open question	What is blocked, and the extension point ready for it
U-12	Are there official use case identifiers in the SRS?	`Specification §10`	Nothing is blocked. The register in Appendix A is derived and labelled as such (D-11); if official numbers appear, only Appendix A changes.

E Diagram index

Figure	Title	Section	Lab step
1	Component and architecture diagram	§2.2	Step 2 — Architecture diagram
2	UML class diagram	§3.1	Step 3 — Class diagram
3	UC-01 sequence diagram	§6.1.1	Step 4 — Sequence diagrams
4	UC-02 sequence diagram	§6.1.2	Step 4 — Sequence diagrams
5	UC-03 sequence diagram	§6.1.3	Step 4 — Sequence diagrams

Lab step	Where it is answered in this document
Step 1 — Methodology & architecture	§2.1, both comparison tables and the three-to-five sentence justification, plus the microservices revisit triggers.
Step 2 — Architecture diagram and components	Figure 1 in §2.2; component responsibility table in §2.3.
Step 3 — Class diagram and classes	Figure 2 in §3.1; class responsibility table in §3.2; design decisions in §3.4.
Step 4 — Sequence diagrams	Figures 3, 4 and 5 in §6.1, linked to UC-01, UC-02 and UC-03 from the Appendix A register.
Step 5 — Design patterns	§7.1, five patterns each with a forcing requirement, plus §7.2 alternatives and §7.3 the decision register.
Step 6 — Validation and traceability	Appendix B checklist, §8 traceability matrix, Appendix C review log.
Step 7 — Assemble the SDD	This document, in the IEEE 1016 structure.